# A well-ordering proof for BTBMS

Under the existence of $0^{\#}$

The G research programme

Concise edition, 26 October 2026

**Abstract**

Let $\mathscr{U}$ be the finite expressions reached from the finite branches of BTBMS Limit by its original fundamental sequences. Working in $\mathsf{ZFC}$ + "$0^{\#}$ exists", we give a well-ordering argument for the original recursive comparison on $\mathscr{U}$. Finite completion, graded ancestor control, complete-window comparisons and simultaneous Point construction yield a strict decrease from every current legal labelling. A fixed finite-constructor-first order on $L$ provides canonical maps and certificates at the actual reflection levels, including weak bare endpoints. Silver indiscernibles supply all finite initial towers below one fixed bound. Least maximum labels, together with a separate finite reachability argument, give an order-preserving ordinal rank. The additional Limit vertex may be ranked by the third Silver indiscernible.

*Editorial note.* This revision replaces the accumulated research appendices with the general arguments used by the proof. It incorporates the three amendments on work models, empty tail prefixes and generated restrictions. The initial G approach and research constraints were supplied by the user; the subsequent arguments and this edition involved substantive ChatGPT assistance. This remains a research manuscript, not a proof-kernel verification or independent peer review.

## Contents

**1 The native system and the theorem** **2**

**2 The original completion** **3**

**3 Global finite invariants and strict barriers** **6**

**4 One fixed canonical system and its low certificates** **12**

**5 Legal labels and the actual work models** **18**

**6 Common Point supply and installation** **20**

**7 Every native step lowers an arbitrary legal source** **25**

**8 Uniform initialization below the third Silver** **29**

**9 From finite paths to the original well-order** **30**

**Source notes and attribution** **33**

**A The pinned native operation** **34**

# 1 The native system and the theorem

## 1.1 Finite syntax and actual occurrences

A finite expression is a finite sequence of columns; a column is a finite sequence of entries; an entry consists of a natural-number parent address and a finite expression called its private height. An entry with empty private height is *bare*. An empty top-level column is allowed. Private columns in an actual nonempty height are nonempty. The notation $p^H$ displays an entry with parent $p$ and height $H$; the superscript is omitted when $H$ is empty. Displayed parent addresses are one-based, while the computational definition uses zero-based addresses.

Addresses refer to the visible column stack. On entering an entry's height the previously visible columns remain visible, the new private columns are added, and they cease to be visible on leaving that private height. Consequently equal integers in different private environments need not identify the same column occurrence. The proof always uses actual occurrences and their binding environments. The *owner* of an entry is its actual containing column. The order of fields is depth first: column, entry, the whole height of that entry, the next entry, the next column. This order is used both for finite positions and for strictly increasing ordinal labels.

The comparison of expressions is recursive lexicographic comparison: first compare columns, then entries in a column, and in an entry compare its parent address before its private height. Proper finite prefixes are smaller. Relative heights retain a flag distinguishing an external parent from an internal offset. Relative comparison compares this flag, the address, and then the height. In particular, a normalization must not discard the flag or identify unrelated private addresses.

## 1.2 The fundamental sequence operation

The native operation $M \mapsto M[n]$ is the operation in the pinned definition reproduced in Appendix A. Its comparison and normalization operations have the meanings just specified. For clarity, its three finite branches are as follows.

1. An expression ending in an empty top-level column loses that column. The empty expression is terminal.
2. Follow the last-entry, last-height path to the deepest bare entry. If its column is a singleton and its parent is the immediately containing owner, use the special rule: delete that singleton private column and repeat the enclosing entry with the truncated height. The exact zero-index and first-copy convention is the one in `expand_special`.
3. Otherwise let $r$ be the root address, $t$ the tail address, and $U$ the tail column before its last bare entry. Compare the whole relative-height word of $U$ with the root column, deleting a smaller root head, a smaller target head, or both equal heads. Keep the unconsumed root suffix. For each of the $n$ copies, copy the bad part with the address shift by a multiple of $t - r$, append that root suffix after the shifted $U$, and substitute the next copy into the distinguished rightmost hole. The root-suffix top parents are kept; its private addresses are shifted. If the root is the final ancestor column, substitute the hole into its final private height instead of inventing a new outer window.

The distinction in the last sentence is essential for ancestor roots. The finite loop in the appendix is an unambiguous recursive definition, including the case $n = 0$; it is not a bounded experimental approximation.

Let $I_k = \mathrm{Limit}[k]$. Here $I_0$ is a single empty column, $I_1 = ()(1)$, and higher $I_k$ are the finite singleton towers specified by `infinity_FS`. Define

$$\mathscr{U} = \bigcup_{k<\omega} \{M : I_k \longrightarrow^* M \text{ by a finite native history}\}.$$

The theorem concerns this actual domain, not every parseable or merely scope-correct finite expression.

**Theorem 1.1.** *Assume* ZFC *and the existence of* $0^{\#}$*. The original recursive comparison well-orders* $\mathscr{U}$*. There are ordinals* $g' < \iota_2$ *and a function* $r : \mathscr{U} \to g'$ *such that*

$$K < M \implies r(K) < r(M), \qquad r(M[n]) < r(M)$$

*whenever the displayed native step is nonterminal. Here* $\iota_0 < \iota_1 < \iota_2$ *are the first three Silver indiscernibles. The additional vertex may be assigned* $r(\mathrm{Limit}) = \iota_2$*.*

We prove this by constructing legal auxiliary labels for the original completed tables. Sections 2–7 establish the one-step construction, Section 8 supplies its initial inputs, and Section 9 passes to the original order.

## 2 The original completion

Throughout, $\nu(H)$ counts columns and entries recursively, while $\mu(H)$ counts entries only. These are sizes of standard templates, not numbers of physical support rows. We write $P \preceq_{\mathrm{pref}} H$ when $P$ is obtained by recursive truncation along the rightmost branch.

### 2.1 Standard representatives, Cut, and scans

A physical completed table contains native entries and virtual support entries. Each occurrence has a standard birth template. Before computing a transported candidate, remove only relative virtual support with its actual birth/deletion witness, and then normalize with respect to the same external owner. A wholly virtual copied package still has a nonempty relative template; it is not erased merely because its global flags are virtual.

For a visible root $r$, $C_r(H)$ is the least height legal at that root which is at least $H$. If transport to $r$ has an illegal external parent, Cut changes the first such parent to the new owner, clears that entry's height, and deletes the later recursive tail. It does not renormalize each inner layer with a different external owner. The finite first- difference comparison gives

$$H \le C_r(H), \quad H \le K \implies C_r(H) \le C_r(K), \quad C_a C_b = C_a \quad (a < b).$$

Indeed a legal height smaller than the cut candidate agrees with the original height until the first illegal field and must be smaller there, or would retain that illegal field. This proves minimality; monotonicity and composition follow from the nested legal classes. All comparisons in this statement use one fixed external owner.

A request scans a root column against the whole target prefix up to the requesting entry. With current heads $R, T$, a smaller root is consumed alone (rootOnly), equality consumes both (both), and a larger root consumes the target alone (targetOnly). A tag records continuing strict coverage. Whenever a new coverage would begin, the root must equal the minimal legal candidate $C_r(T)$; if that candidate is missing it is inserted. An insertion before a root head has the same parent and a strictly lower height; a tail insertion is parentless. The final actual consumer is $\Gamma$ of the request. All requests, including virtual and private ones, are rescanned on the completed physical table.

**Lemma 2.1** (Consumer preservation)**.** *In a qualified finite scan, insertion of a lower root immediately before an existing root, or insertion of a lower target before its existing right carrier followed by completion of its own needs, preserves the actual consumers of all old targets.*

*Proof.* For a root insertion $V < R$, an old target that skipped $R$ also skips $V$. If $R$ began a nonempty consumer block at $T$, then $C_r(T) = R$; minimality forces $V < T$, so $V$ is skipped before

that block. A tail insertion is beyond the old scan. For a target insertion $T' < T$, let $G$ be the old consumer of $T$. Earlier roots consumed or passed by $T'$ are still passed by $T$. If $G$ was already covering, it remains. If coverage was not active, $G = C_r(T)$ and $C_r(T') \leq G$. Equality opens coverage without removing $G$; strict inequality introduces a candidate below $T$, which $T$ skips. In every case $T$ reaches the same actual $G$. Iteration proves the assertion. □

**Theorem 2.2** (Finite completion)**.** *Fairly completing all actual requests makes only finitely many insertions. The result includes completed private packages and virtual requests.*

*Proof.* Let $D_x$ be the standard template at the birth of an actual entry $x$. Every later support inserted before a head has that head as its strict same-parent deletion witness; a tail insertion has the parentless deletion witness. Induction on the finite insertion history and then on private containment shows that support reduction always recovers the same $D_x$. Thus $n(x)$, the recursive number of entries of $D_x$, does not change. The visible root set of $x$ is also fixed; let its size be $q(x)$.

For a fixed pair consisting of $x$ and an actual visible root, at most one candidate is created. Once it consumes $x$, Lemma 2.1 preserves that occurrence as consumer despite all later lower insertions. It cannot be requested anew. Organize creation as a forest: a candidate belongs to its actual triggering target, and each directly private entry belongs to the entry whose birth template contains it. Along a candidate edge $(n, q)$ decreases lexicographically because Cut does not increase $n$ and the root is earlier. Along a private edge $n$ strictly decreases. Every vertex has finite branching. Induction on $(n, q) \in \omega^2$ therefore proves that each creation subtree is finite. There are only finitely many initial roots. Every individual scan also consumes a head except when it creates a candidate that immediately consumes the pending target. Fair rescanning consequently terminates. This uses well-foundedness of natural-number pairs, not of BTBMS expansion. The same induction gives a recursive bound by summing the finitely many smaller creation subtrees. □

## 2.2 Finite scan identities

It is useful to distinguish the comparison of finite words from the insertion of support. For words $R, W$ over any linearly ordered alphabet, let $\mathrm{res}(R, W)$ be the root suffix left by the three comparison rules: delete the smaller head, or both equal heads, and stop when one word is empty. A missing root leaves no residual parent. All parent chains below are stopped at that point or at a parentless head.

**Lemma 2.3** (Absorption and residuals)**.** *Scanning successive target words composes:*

$$\mathrm{res}(R, UV) = \mathrm{res}(\mathrm{res}(R, U), V).$$

*If $x < y$, the target word $xy$ has the same action as $y$ on every root word. Consequently every target word has a unique weakly decreasing normal form obtained by repeatedly removing a letter that precedes a strictly larger letter. The residual action depends only on this normal form. A lower same-parent root inserted immediately before a higher root does not change the parent of the first residual head.*

*Proof.* The first identity is the scan with its target pointer paused between $U$ and $V$. For $xy$, roots below $x$ are removed in either scan. If the next root equals $x$, consuming it before scanning $y$ has the same effect as removing it as a smaller root during the $y$ scan. If the next root lies strictly between $x$ and $y$, the first target is removed and that root is then removed by $y$. If it is at least $y$, the first target disappears without affecting it. These are all cases.

A stack algorithm gives the normal form: read from left to right, remove terminal stack letters smaller than the new letter, and append the new letter. The stack is weakly decreasing; the

preceding absorption identity proves that its residual action is unchanged. Its letters are exactly the right-to-left weak maxima, with their multiplicities, so it is independent of the order of removals.

For the last assertion compare the scan with and without the inserted root $V < R$. If the two residual suffixes differ, one starts at $V$ and the other at $R$. Their first parents agree. If $V$ is removed, the pointers agree again; if a target removes $R$ by equality, it has already removed $V$. This proves parent neutrality, not equality of the two residual words. Repeating it handles a finite virtual group before one right carrier. □

In particular, with 0 the empty height, if scanning $W$ against $U0$ leaves exactly its final 0, then

$$\mathrm{nf}(U) \le \mathrm{nf}(W) < \mathrm{nf}(U)0, \qquad \mathrm{nf}(W) = \mathrm{nf}(U). \tag{2.1}$$

The equality follows because no weakly decreasing finite word lies strictly between a word and that word followed by the least letter. This identity handles a bare edge newly created by Cut. It does not require the letters of $U$ to be bare.

For completed scans, record also a Boolean tag. A strict targetOnly event begins a coverage block only at the least legal candidate; further targets may be consumed by that same root while the tag persists. A rootOnly or nonfinal both event resets the tag. This is a statement about actual positions in the completed word, so repeated equal heights are not identified as one occurrence.

**Lemma 2.4** (Composition of qualified scans)**.** *Suppose the legal root classes are nested, and their least-candidate operators satisfy $C_aC_b = C_a$. For three compatible completed prefixes, the scan from the first to the third is the composition of their actual consumer maps through the second. If the two endpoint heights are equal, every segment of the corresponding consumer chain is a both event.*

*Proof.* Partition the middle word into consecutive blocks with one consumer in the first word, and the last word into consecutive blocks with one consumer in the middle word. Follow the first unprocessed target in the last word. Let $B$ be its middle consumer and $A$ the first-word consumer of $B$. Every first-word root before $A$ has either been removed by an earlier middle target or is strictly below the first target at which its block could start. It is likewise removed or passed in the composed scan. If $A$ opens coverage, minimality gives $A = C_a(C_b(T)) = C_a(T)$ at the opening target $T$. Thus no earlier legal root can replace $A$.

While the middle consumer remains $B$, the same $A$ continues to cover the corresponding last-word block. When $B$ changes, compare the new middle head with $A$: a smaller head remains in the block, equality ends it by both, and a larger head removes $A$ before the next block. These are exactly the three direct-scan events. This is a finite induction on the two moving pointers, and does not require either word to be sorted. At every consumption the root height is at least the target height. If endpoint heights agree, every intermediate inequality is an equality. The actual events are therefore both, as asserted. □

The equality is about completed compatible prefixes. Removing a root that is still needed by a retained target destroys its premise. Conversely, after deleting an unused virtual package, replay and minimal support in Lemma 2.5 re-establish the same premise. This is why support pruning is performed through birth records and actual consumers, rather than by erasing all virtual entries at once.

## 2.3 Physical identity and pruning

Finiteness alone does not identify an arbitrary qualified table with the original completion. The required identity is stronger.

**Lemma 2.5** (Replay and minimal support)**.** *Fair executions of the original insertion rules produce the same physical completed table. A completed truncated package is the original completion of its native projection if its retained whole packages have their original creation records and its retained support is the least actual* $\Gamma$*-closed support of the retained native packages.*

*Proof.* Use one finite completed execution as reference. A partial replay has only gaps bounded by the strict right carriers from that execution, or a parentless tail gap. At the first missing required consumer, the least-candidate rule forces the same reference occurrence; lower interposed support cannot change old consumers by Lemma 2.1. Induction embeds any other original execution in the reference creation record. If a proper completed subtable remained, take the first omitted creation. Its driving target and all earlier prerequisites would already be present, so its required consumer could not be omitted. This proves equality.

For a truncation, replay from its native template with the retained external ports. The same first-gap argument keeps the replay inside the proposed support. Whole-package closure follows by replaying each retained package's internal creation history; unrelated outside requests cannot enter its private columns. The replay contains the native packages and is closed under their actual consumers, so minimality of the proposed support gives the reverse inclusion. At a final non-tail gap the retained right carrier is essential: it certifies the strict deletion relation of every interposed support. At a tail gap the corresponding certificate is parentlessness. □

# 3 Global finite invariants and strict barriers

## 3.1 Graded ancestor control

For a standard height put

$$d(\varnothing) = 0, \qquad d(H) = 1 + d(H[0][0].\text{height}) \quad (H \neq \varnothing).$$

This is the depth of its first branch, not its maximum tree depth and not the depth of a physical first support row. Define the following finite properties.

**F.** Every direct private column of an owner $q$ begins with an entry whose parent is at least $q$, recursively.

**G.** If an entry $u$ reads an ancestor column $r$, and $t$ in $r$ contains $u$, every height in the target prefix through $u$ has first- branch depth strictly below $d(\text{ht}(t))$.

**EA.** Let $U$ be a nonempty column prefix whose last parent is $p$, and let $a$ be the next entry's parent. Write $\pi_W(c)$ for the parent of the first head remaining after scanning the whole word $W$ against column $c$. Then $a$ belongs to the chain $p, \pi_{\mathcal{H}(U)}(p), \pi^2_{\mathcal{H}(U)}(p), \ldots$.

The first parent of a first private column is its owner. Comparing first branches therefore gives

$$d(H) < d(K) \quad \Longrightarrow \quad H < K. \tag{3.1}$$

Cut preserves the first branch of each retained nonempty containing height: its first illegal external parent cannot be the first entry of a private column. The first difference is reached before a later illegal parent can change that branch.

**Theorem 3.1** (Finite-history invariants)**.** *Every* $M \in \mathscr{U}$ *and its original completion satisfy F, G, and EA. In a request reading an ancestor column, every actual consumer lies strictly before the containing entry in that ancestor. If a target* $s$ *is consumed at a root* $r$*, then* $\Gamma^m(s) = r$ *for some finite* $m > 0$*. Equality of their heights makes every segment of this chain both.*

*Proof.* The singleton towers satisfy F and G directly and have no nontrivial adjacent EA requirement. Use induction on the length of the actual finite native history, with the scan identities proved in Section 2.2.

Closed copies preserve parents, depths and comparisons. At a new general seam the preceding shifted $U$ is followed by $V = \mathrm{res}(R, U)$. If an entry of $V$ reads an ancestor whose containing head has depth $g$, its old complete root prefix had depth below $g$. A letter of $U$ of depth at least $g$ would already have consumed that prefix; hence such an entry could not remain in $V$. This proves G across the only new seam. Nonfinal holes retain their corresponding first branches. A zero final hole can lower a first branch only along a singleton-column, singleton-entry chain; the containing depth and the internal prefix depth shorten together, preserving the strict gap. Special repetition has the same shortened chain and adds only same-parent adjacency. This proves F and G without assuming EA of the output.

For EA, transport each finite threshold word through the inverse comparison map of the particular copy currently being considered. If the scan exhausts $U$, the residual-word identities give the old root's remaining parent. If it stops in $U$, old EA moves that parent to the preceding copy boundary. The copy number decreases. On meeting an open ancestor, G makes the threshold word strictly lower than the containing head, so the scan never reads the changing tail inside that head. Queries inside a closed $U$ package stop at their own containing head before the appended $V$; this prevents a circular appeal to the seam currently being proved. The final zero hole has no later copy reading its changed tail. These cases exhaust retained prefixes, $U$, the seam, $V$, closed bad-part copies and private queries. Zero and special steps use the same ancestor bound and same-parent zero-length chains. This proves EA under native steps by ordinary induction on finite history length.

For completion, a lower same-parent insertion is neutral for the parent of the first residual head: scanning it followed by its higher right carrier has the same absorption effect as that carrier. Its depth is no greater than the carrier's. Its private Cut is legal at the same common ports. Birth-record induction thus preserves F, G and EA under every insertion. Completion is finite by Theorem 2.2.

Finally qualified scans compose their actual consumer maps. Partition targets into consecutive blocks consumed by each root. Minimality and $C_a C_b = C_a$ identify the first consumer of every composed block; equality and the next strict inequality identify its end. No sorting of the words is assumed. Apply this composition along EA, inducting on actual owner and request position. It gives the stated $\Gamma$ chain, including targetOnly terminals. Heights cannot strictly fall on a chain whose endpoints are equal, so such a chain consists of both events. At an ancestor containing head, (3.1) and G make the entire target and its minimal candidates strictly lower. A fresh coverage must therefore insert its candidate before that head, so that head cannot be an actual consumer. □

### 3.2 The candidate menu and actual barriers

For $g > 0$ define the common-port set

$$\Pi_g(q) = \bigcap_{\substack{W \neq \varnothing \\ \max d(W) < g}} \mathrm{Anc}_{\pi_W}(q),$$

excluding the starting point $q$. The words may include formal letters realizing a finite comparison pattern at the same depth; these are proof devices, not installed BTBMS entries. If an actual entry of height-depth at least $g$ reads from $q$, EA and G put its parent in $\Pi_g(q)$. Cut at such a port preserves F, G and EA, because every exit through $q$ is along the very same threshold-word chain through the new port. Different words' chains are not spliced together.

**Lemma 3.2** (Candidate dichotomy)**.** *For an actual active height $H$ replaced by its retained result $P_n$, and an actual backward query root $r$, the candidate is*

$$C_r(P_n) \in \{P_n, C_r(H)\}. \tag{3.2}$$

*Proof.* Fixed old illegal cuts and zero truncations are decided before the changed suffix. For a general positive step there are two new boundaries. If the first illegal field is the last bare parent $b$, the preceding word $U$ is nonempty and has depth below $d(H)$. Both $b$ and $r$ lie on its one residual-parent chain. For $b > r$ the new residual parent still lies in $[r, b)$, so the old Cut is restored. For $b = r$, the residual root parents are all below $r$ and the shifted private fields are internal, so $P_n$ is exactly legal. If $H$ was already legal and the mother root is the owner $q$, G gives the same low-depth word, and $r \in \Pi_{d(H)}(q)$ makes its first residual parent lie in $[r, q)$; Cut again restores the old owner-bare boundary. Internal roots create no new illegal external parent, and an external legal root was already below $r$. Special positive steps delete an internal singleton owner-bare column and do not change the first illegal external field. These cases prove (3.2). □

The strict-barrier assertion used below is restricted to actual corresponding scans: if a selected carrier satisfies $P < B < H$, then $P$ is a recursive prefix of $B$. It is not asserted for every height in an arbitrary numerical interval. The proof uses complete suffixes, the first actual origin of a strict carrier, and the same actual root window. For an internal origin the repeated native window bounds the entire suffix between two iterates, and recursive-prefix convexity gives the assertion. For an external origin the common-port chain identifies the same root; filling its complete window would give $T_n \le X < \Psi(X) \preceq T_1 \le T_n$, a contradiction. Later barriers inherit the first barrier's prefix by interval convexity. Sections 3.3–3.4 give the complete-window and source arguments.

### 3.3 Complete windows along finite histories

Fix an actual nonempty column prefix $U$ in a column $q$, let $p$ be its last parent, and let $s$ be the next parent, or the parentless sentinel if there is no next entry. A *qualified cut* consists of a root $r > s$ on the single residual-parent chain of $p$ for the entire word $\mathcal{H}(U)$. Let $X$ be the complete same-level suffix beginning at this cut. It includes all later columns and their private heights.

Temporarily truncate to $U$ and append a formal bare entry with parent $r$. Apply one native copy window while leaving its rightmost recursive hole unfilled. This defines a one-hole context $\Psi$. It retains $U$, appends the complete root residue, shifts addresses by the native threshold, and fills the hole by its argument. If $r$ contains $q$, property G makes the whole threshold word strictly lower than the containing head. Hence the scan stops before that head is consumed, and the ancestor branch of the native formula still defines the same one-hole context. The formal context is only used for comparison; it is not asserted to be an actual node.

The hole follows a nonempty finite constructor. Both ordinary and ancestor contexts strictly preserve order and recursive prefixes. Define $T_0 = \varnothing$ and $T_{j+1} = \Psi(T_j)$.

**Lemma 3.3** (Finite context criterion)**.** *The $T_j$ form an increasing chain of recursive prefixes, and for each finite complete suffix $X$,*

$$X < \Psi(X) \quad \Longleftrightarrow \quad (\exists j \ge 1)\ X \le T_j. \tag{3.3}$$

*Proof.* The empty suffix is a prefix of $\Psi(\varnothing)$; prefix preservation proves the first assertion. If $X \le T_j$, compare $X$ with $\Psi(X)$ at the first place where $X$ leaves the common iterated framework. Leaving earlier gives $X < \Psi(X)$, while staying in the framework reduces the comparison through the hole. After at most $j$ reductions one reaches the empty suffix, where strictness holds. Conversely, suppose all $T_j < X$. The first difference between $X$ and each $T_j$ must lie after the framework shared with the preceding iterate; otherwise strict order preservation would contradict $X < \Psi(X)$ at that same difference. Each reduction passes a nonempty constructor. Eventually the common

framework would pass the last field of finite $X$, which is impossible. This is induction on the finite first-difference path, not on BTBMS descent. □

**Lemma 3.4** (Native window transfer)**.** *For corresponding cuts in copies of the same mother step, let $\sigma$ be their actual address transport and suppose its threshold $a$ satisfies $r \le a$ for the fixed root $r$. With the native remaining-copy indices,*

$$\sigma\Psi_i(X_i) \preceq_{\mathrm{pref}} \Psi_j(X_j), \qquad X_j \le \sigma X_i. \tag{3.4}$$

*Proof.* Pull the finite comparison word back separately in each copy. Before an ancestor is met, closed heads are compared by the same order-preserving map. At the ancestor, G stops the scan before its changing tail. After the first transport, the root's private addresses are internal, and the second shift adds to the first. True external parents and the top parents of the root residue remain fixed. Thus field-by-field transport commutes with filling the complete hole; the possible loss is only a right truncation. This proves (3.4).

If the source mother root itself is open, copy $j$ has only $n - j$ mother iterations left. Using $n$ would fill the wrong hole. There is also a separate owner case: if the mother tail is $t$, the root is $b$ and $d = t - b$, then the $j$th transported tail-prefix package has owner $b + (j + 1)d$, not $b + jd$. A query to its own containing owner uses the closed containing package and commutes with transport. A query to an earlier boundary either leaves a head in the installed prefix, in which case its first residual parent is already strictly above the next parent $s$, or exhausts that prefix and returns to the mother's residue. In the latter case apply the residual identity of Lemma 2.3 and the earlier-boundary instance of (3.4). The boundary index decreases. This treats owner queries without applying (3.4) at an ineligible threshold below its fixed root. □

**Proposition 3.5** (Actual complete-window bound)**.** *Every qualified cut of every $M \in \mathscr{U}$ satisfies $X < \Psi(X)$. Every standard completed candidate satisfies the corresponding statement for roots internal to that candidate.*

*Proof.* Induct on the length of the given finite native history. In an initial tower, a cut lies at the end of a singleton column. Its suffix is already a prefix of the first nonempty window, so Lemma 3.3 applies. For the induction step classify the position of the cut in the explicit native construction.

1. At a retained prefix $U$, closed earlier roots and the formal window are unchanged. An open containing root is protected by G. The suffix only decreases, so its old finite-iterate bound remains. If the last entry of $U$ changes internally, the relevant level has the single retained column $U$, which is a prefix of $\Psi(\varnothing)$.
2. At the first new boundary, EA puts the old next parent and the new residual parent consecutively on the same chain. The only new qualified root between them is the mother root. Its new suffix is the actual $T_n$, giving the required finite bound directly.
3. A cut in the root residue uses the same normal form of the complete scanned prefix. Exhausting an installed prefix composes the same one-hole context. If its final retained entry changes, use the single-column case of the first item.
4. A cut in the bad part, a root-private package or a later boundary is governed by Lemma 3.4. A query leaving a head in the installed prefix is decided by the strict first-parent comparison. An exhausting query uses the old residual window and the correct remaining-copy index. Moving to an earlier boundary decreases that finite index.
5. At the final zero hole, the entire deepest last column disappears. A removed private cut has no surviving obligation. A retained cut whose $U$ changes is the single-column case; otherwise its containing head is protected. Special repetitions preserve same-parent adjacency; later copies use the preceding complete copy and the same window fill. A new same-parent seam has no new root satisfying $r > s$.

These cases cover the retained prefix, new seam, residue, bad part, private copies and deleted tail. They use the induction hypothesis only on the actual mother node, not on an arbitrary formal substitution.

For Cut, restrict attention first to an internal root. If Cut changes the last retained entry of $U$, the output has the single-column form already considered. Otherwise all earlier closed fields are unchanged, and G protects every containing head before the cut. The same formal context has legal finite iterates at the destination owner: its retained framework is legal, and copying only moves internal fields. Choose a strict old finite-iterate upper bound. Minimality of Cut bounds the new suffix by that legal iterate, and Lemma 3.3 applies again. Recursive prefixes, true subheights and whole address transport preserve this argument. Birth-record induction therefore gives the assertion for all standard completed candidates. No arbitrary external port is added by this internal version. □

### 3.4 Source ports and strict carriers

There are two transport issues in applying Proposition 3.5. First, equal numerical addresses in unrelated private environments are not common ports. Second, a Cut-created bare edge has to be pulled back to an old chain; it cannot be declared an old edge.

Suppose a whole package at owner $q$ is moved to a compatible port $a$. For a retained non-bare internal entry, transport an arbitrary finite threshold word of smaller depth by its finite comparison pattern. Closed fields before Cut are either true external parents below $a$ or internal fields at least $q$; transport is strictly increasing on this allowed set. The standard heights in a common external column are fixed by the same comparison map. At a containing ancestor G again prevents the scan from reading the changed tail. If the new chain goes through the new owner, the old chain goes through $q$, and the common-port condition supplies the rest of that very same word's chain to $a$. If it uses the new final bare edge, (2.1) identifies its word with the normal form of the cut prefix. Old EA then supplies the missing old segment. If the bare entry is consumed, there is no new residual edge to justify. This proves that every compatible common external port of the moved entry was already a compatible port of its actual source.

Tracing a non-bare head $B$ in an original completed column $r$ through its actual birth record now gives a source occurrence $x$, of height $X$, such that

$$B = C_r(X), \quad d(B) = d(X) = g, \quad r \in \{\mathrm{owner}(x)\} \cup \Pi_g(\mathrm{owner}(x)). \tag{3.5}$$

This traces one actual column, including through private copies; it does not infer provenance from a printed parent number.

New support in the original gap immediately before an active height $H$ has depth below $d(H)$. Indeed a request from within $H$ has that depth bound by G; a request from an earlier closed package terminates before its own containing head and never enters this gap; a request from a package newly inserted in the gap has still lower depth. Induct on creation order. Therefore such support lies below every positive result $P$ with $d(P) = d(H)$ and cannot be a strict barrier.

**Proposition 3.6** (Strict-barrier theorem)**.** *In an actual corresponding backward query for $H \to P$, every original strict carrier $P < B < H$ contains $P$ as a recursive prefix. The same conclusion holds after the permitted preparation, with the four carrier origins specified below.*

*Proof.* Consider first a positive general change with nonempty deepest tail prefix $U$. If the mother root $b$ is internal to $H$, the interval $P < B < H$ forces $B$ to retain the entire framework through $U$, with next parent below $b$. EA therefore leads to the same corresponding internal root: a chain cannot leave the package and later return to an internal column, and G protects any containing head on the route. The two complete contexts agree. The suffix of $P$ is $T_n$; the suffix $Y$ of $B$ has

a finite bound $Y \le T_k$ by Proposition 3.5. Thus $T_n \le Y \le T_k$. Since the iterates are recursive prefixes, prefix interval convexity gives $T_n \preceq_{\mathrm{pref}} Y$, hence $P \prec_{\mathrm{pref}} B$.

If $b$ is external, let $r < q$ be the compatible actual query column. Candidate dichotomy and minimality give $C_r(P) = P$. The owner-boundary cases in that dichotomy exclude $b = q$ and $r < b < q$, so $b \le r < q$. Equation (3.5), followed back to the first strict source, gives

$$P < X \le B = C_r(X) < H.$$

This $x$ is not the active entry or an entry inside $H$. The common framework up to the deleted last bare entry identifies a cut in $X$ with the same complete $U$ and next parent below $b$. For internal pieces, comparison is transported by the same order map. All true external fields of that framework are below $r$ and hence name common actual columns. If the chain leaves the source owner, (3.5) puts both threshold chains through that same $r$; their tails then agree down to $b$. Proposition 3.5 gives $Y < \Psi_x(Y)$ for the complete source suffix $Y$.

Perform only the actual zero deletion at this source cut, and call its remaining complete suffix $J$. The common framework survives: otherwise it would contain the global deepest bare entry and have fewer entries than $H$, contrary to $P < X < H$ at this cut. Put $m = \nu([U])$. Then $J \preceq_{\mathrm{pref}} Y$, and the removed suffix has size either zero or $m + 1$. The fixed part of $\Psi_x$ contains all of $U$ and one additional entry leading to the hole, hence has size at least $m + 1$. For an ancestor root that extra entry survives by G. Filling the full window therefore gives, under the actual source-target field correspondence,

$$Y \le \widehat{\Psi_x(J)} \preceq_{\mathrm{pref}} T_1 \preceq_{\mathrm{pref}} T_n. \tag{3.6}$$

Here both internal shifts add and true external parents remain fixed; one must retain all same-level columns when making this comparison. But $P < X$ gives $T_n \le Y$. All comparisons in (3.6) must be equalities. If $J = Y$, the source strict window bound contradicts equality. If $J \ne Y$, equality leaves no unaccounted field outside the rightmost containing chain and gives $X = P$, also a contradiction. Thus an external root has no strict carrier in this interval.

If $U$ is empty, use the complete-column window: the root is an earlier nonempty direct private column, its entire residue survives the empty scan, and the preceding internal-root argument uses that full column as the nonempty context. Zero changes are recursive truncations, and special changes have repeated identical same-parent constructors; in both cases the first-difference prefix comparison gives the assertion directly. Later strict carriers inherit the first one's prefix by recursive-prefix interval convexity. The four permitted preparation origins are checked in the following paragraphs and Lemma 3.7. □

For the prepared construction there are four possible carrier origins: old heads, returned internal support, stage-0 whole-package images, and arrival blocks. Old heads use the preceding assertion. Required internal support has smaller target size and smaller first-branch depth at its certified query; a larger temporary shadow is instead traced through its actual birth record to the smaller returned package or old source. A stage-0 image can be pulled back along its actual common-port chain, where the native root-window assertion applies. Arrival blocks use the following counting argument, including its empty-prefix case.

**Lemma 3.7** (The empty-prefix interval case)**.** *Let $H \to P_n$ be a general positive change in a legal private height. If $P_n < B < H$, then the recursive entry count satisfies $\mu(B) \ge \mu(H)$. Therefore a carrier originating in a proper internal height of $H$, followed by Cut and transport, cannot lie in that interval.*

*Proof.* When the deepest tail prefix $U$ is nonempty, remove the final bare entry and call the resulting common recursive prefix $E$. Then $\mu(E) = \mu(H) - 1$ and $E \preceq P_n, H$. When $U$ is empty, delete its whole singleton-bare column; the same entry-count equality holds. By F the last bare parent is at least the private owner. Equality would make this a special step, so the present

general root is an earlier nonempty direct private column. Scanning it against empty $U$ leaves a nonempty root suffix. Every positive step inserts a nonempty column at the deleted position, and $E \prec P_n$.

Recursive-prefix interval convexity now gives $E \prec B$ in both cases. At this private position a strict extension adds an entry, since a private column cannot be empty. Hence $\mu(B) \geq \mu(E)+1 = \mu(H)$. An internal arrival source $\eta$ instead gives $\mu(B) \leq \mu(\mathrm{ht}(\eta)) < \mu(H)$, a contradiction. The source inequality also holds for a virtual arrival head by tracing its genuine birth record. Counting columns as well as entries would remove two objects in the empty-$U$ case and restore at least two; one must not retain the entry-count "minus one" formula in that different convention. □

These arguments prove the strict-barrier hypothesis for all four actual source classes. Connection records must first be followed to their actual Init source; an arbitrary intermediate connection head is not substituted for the original query.

# 4 One fixed canonical system and its low certificates

## 4.1 The order and the operations

Unless explicitly discussing the wider convention, "positive elementary" means elementary for formulas $\exists y\, \delta$ with $\delta$ bounded, and their finite tuple versions. Write this class as $\mathcal{E}$. This fixed convention is used both for incoming maps and for natural D4 inclusions.

At every passage from $L_\alpha$ to $\mathrm{Def}(L_\alpha)$, retain the old order and order new objects by their earliest definition code. The codes have two successive blocks. The first block consists of the literal finite-set constructors

$$\chi_m(x; y_0, \ldots, y_{m-1}) \equiv x = y_0 \vee \cdots \vee x = y_{m-1},$$

with the false formula for $m = 0$, ordered first by $m$ and then lexicographically by the old order on parameters. The second block contains the other first-order formulas in a fixed syntactic order, then their parameter tuples. At limits take the union of the earlier orders. Denote this one global order by $<_L^{\mathrm{fin}}$.

For each existential bounded formula, select the least *complete witness tuple*, in its fixed ordinary set-theoretic encoding, and project to the prescribed output. This defines the fixed canonical operations and their finite-term hull $\mathrm{Hull}_{\mathrm{fin}\,t}(A)$ in $L_t$. Define

$$\mathrm{Gen}_{\mathrm{fin}}(p,t) \iff L_t = \mathrm{Hull}_{\mathrm{fin}\,t}(p+1), \qquad \mathrm{Cut}_{\mathrm{fin}}(p,c;t) \iff \mathrm{Hull}_{\mathrm{fin}\,t}(p \cup \{c\}) \cap c = p.$$

Here $p+1$ includes $p$ itself; $p \cup \{c\}$ contains all ordinals below $p$ and $c$, not $p$ as an additional generator. The graph used for a request is the inverse transitive collapse of this specified hull, with its required exact source. The order is never changed during reflection.

## 4.2 Coding a weak finite successor level

Write a nonlimit endpoint as $t = \lambda + n$, with $\lambda > \omega$ limit and $0 < n < \omega$, and put $A = L_\lambda$. Finite codes in $A$ describe the objects in each of the $n$ successive Def layers. At a successor, a code specifies a formula and finitely many earlier codes; equality and membership are checked using the preceding layer's correct satisfaction relation. Finite assignments are coded flatly, so their overhead does not grow with the number of coordinates.

There is a fixed finite overhead for each of the layer-identification, finite-evaluation and finite-truth constructions. A limit absorbs this overhead. Thus the required fields can be separately supplied at the next level; their ordinary joint tuple need not belong to that level. Correctness is proved externally by induction on the actual Def construction, with bounded verification of each

recursive step. The coding construction below keeps truth for proper layers separate from truth for the endpoint itself. No assertion that $\mathrm{Sat}(L_t)$ always belongs to $L_{t+1}$ is made.

### 4.3 Layer identification and partial truth

Here and below, a certificate means a tuple of specified objects with a stated check, not an object whose correctness is merely asserted. When a boundary lacks pairing closure, the objects are quantified separately. Codes for finitely many coordinates are flat finite sequences over a limit base; they are not the ordinary high sets whose canonical order will later be compared.

**Lemma 4.1** (Correct small layers)**.** *There is a bounded check for a proposed constructible layer $L_\alpha$ using its proper hierarchy diagram, finite assignment domain and proper layer truth. It is sound in every transitive environment containing the data and the true $\omega$. Each required diagram has a fixed finite coding overhead. At a limit, this overhead is absorbed below that limit for each proper layer.*

*Proof.* Supply a function $H$ with domain $\alpha$, whose value at $\beta$ is claimed to be $L_\beta$; do not include the endpoint as a self-referential pair. Require the zero clause, union at limits, and at successors the two directions of the Def clause. The latter say that every displayed new set is defined by a formula with finite parameters over the previous layer, and that every such definition is represented.

Supply also a domain $Q$ of finite assignments and a truth relation $T$ for those proper layer structures. Check the empty assignment, closure under appending one member of the relevant structure, and the finite length function. External induction on the true natural numbers proves that every actual finite assignment occurs. Atomic clauses use actual membership and equality. The negation and Boolean clauses are bounded, and a quantifier ranges over the displayed small structure. External induction on formula construction therefore proves that $T$ is its actual satisfaction relation. Induction on $\beta$ now proves the hierarchy diagram and both directions of Def. Thus a new spurious hierarchy cannot pass the check.

The data may be encoded as flat membership-incidence records indexed by formula numbers, finite positions and structure elements. Pairing depth is then fixed, rather than growing with formula arity. A finite subformula table for a fixed formula is obtained by finitely many Def constructions; its full relation can be specified as one definable incidence table. The hierarchy and truth diagrams are constructed together by external induction, with one fixed finite reserve for these flat records. At a limit $\lambda$, any particular proper-layer construction has $\beta + N < \lambda$ for a finite $N$. This last assertion does not place the endpoint's full truth in its own layer or in its immediate successor. □

Let $M_r = L_{\lambda+r}$ and $A = L_\lambda$, for one fixed actual $r < \omega$ and limit $\lambda > \omega$. A code in $A$ specifies a finite tree of Def descriptions. Its evaluation $\mathrm{ev}_r$ is onto $M_r$: an element of $A$ codes itself, and at each successor every definable subset has a formula and finitely many earlier parameters. All finite lists of these codes belong to $A$. Evaluation and the atomic relations have correct definitions using the proper layer structures. In an external structure containing $M_r$, relativizing those definitions to $M_r$ gives bounded checks.

**Lemma 4.2** (The tight truth tables)**.** *The coded equality and membership relations, and coded truth for $\Delta_0$ and $\exists\Delta_0$ formulas over $M_r$, belong separately to $M_{r+1}$. Their correctness has a bounded check on the code domain. For each fixed first-order formula, its finite subformula truth table also belongs to $M_{r+1}$. These assertions do not put one total first-order satisfaction table for $M_r$ in $M_{r+1}$.*

*Proof.* For $r = 0$, choose a proper transitive layer containing the finitely many parameters of a bounded query. Its correct layer and truth records fit below $\lambda$ by Lemma 4.1. Bounded absoluteness

decides the query. Both truth and falsity have such finite positive certificates. Existential witnesses can be specified by finitely many codes in $A$. The relations are therefore definable over $A$, and their tables belong to $\mathrm{Def}(A)$.

For $r > 0$, expand each top-layer object into its definition over $M_{r-1}$. A variable bounded by such an object ranges over $M_{r-1}$. Translate equality and membership by substituting those definitions. A fixed bounded query becomes a fixed first-order formula over $M_{r-1}$. The finite table of this formula and its subformulas belongs to $\mathrm{Def}(M_{r-1}) = M_r$. A finite syntax trace verifies the translation. Truth and falsity of the original query are thus definable over $M_r$. For a narrow positive formula, additionally quantify over the finite codes of its witnesses, and use the same bounded matrix table. This proves the next-level membership of the two partial tables.

For the external check, first verify the atomic code relations by relativized evaluation. The Tarski clauses for all bounded formulas quantify only over $\omega$, the code domain $A$ and the supplied relations. A bounded variable ranges over those codes whose evaluated values belong to its bound. The narrow positive table is checked by existence of witness codes in $A$ followed by the bounded table. These are bounded quantifiers over fixed sets. Since evaluation is onto, the checks omit no actual assignment. External syntax induction proves their soundness for every candidate table.

For an arbitrary fixed formula, use the same finite-subformula construction. Its finitely many relations are definable over $M_r$; their flat incidence table belongs to $M_{r+1}$. The statement is formula-by-formula. Collecting them all as one total truth set would require a further layer and is not part of the lemma. □

For clarity, full coded satisfaction over $M_r$ belongs to $M_{r+2}$: in $M_{r+1}$ it is defined by existence of the correct finite table for the queried formula, and a further Def step collects the relation. Thus the full truth of $M_{r-1}$ needed to identify $M_r$ lies in $M_{r+1}$. Whenever $\lambda + r < H$, this level is contained in $L_H$. At a limit $\lambda$, the proper hierarchy and truth diagrams themselves are separately definable over $L_\lambda$ and belong to $L_{\lambda+1}$. These are the endpoint bounds used in the next lemma.

These distinctions give two exact elementarity checks. For the narrow class, supply the correct partial tables at source and target and check that the proposed graph preserves their values on all parameter codes. For the wider class $B\Sigma_1$, one can instead use

$$\forall i \in \omega \ \big(i \notin \mathrm{Code}(B\Sigma_1) \ \vee \ \exists R_s, R_t \, [\mathrm{FinTruth}_s(i, R_s) \wedge \mathrm{FinTruth}_t(i, R_t) \wedge \mathrm{Match}(i)]\big). \tag{4.1}$$

Each $R$ is the correct finite table for that formula. Formula (4.1) is in the bounded positive closure; it must not be silently prenexed into $\exists\Delta_0$ in a weak layer. A supplier used with this convention must preserve that same class. If the narrow convention is used throughout, the partial-table check already has the required form.

**Lemma 4.3** (Actual maps preserve proper layers)**.** *An actual positive elementary map $J : L_H \to L_V$ satisfies $J(L_\alpha) = L_{J(\alpha)}$ for $\alpha < H$. Its restriction to each proper layer is fully elementary. If its critical point is $p > \omega$ and $J(p) = c$, then $p, c$ are limits and $J$ fixes $L_p$ pointwise. In particular $L_p \prec L_c$ is the natural full elementary inclusion.*

*Proof.* Transport a correct proper-layer identification and its relative formula records through $J$. Bounded soundness identifies the transported layer as the actual constructible layer, not merely an isomorphic candidate. Equivalently, its proper hierarchy and Def clauses are characterized by the correct small-layer diagram. For a fixed formula $\varphi$, relativize every quantifier to the set $L_\alpha$. This is a bounded formula in that set and its parameters. Positive elementarity therefore gives full elementarity of the actual restriction.

A least moved ordinal above $\omega$ cannot be a successor, because successor is definable and its predecessor would already move. The same relative characterization shows that its image is a limit. Induct on $\beta < p$: lower layers are fixed; every subset at a Def successor has a formula and finite fixed parameters, so the defined set is fixed; unions handle limits. Thus every member of $L_p$

is fixed. Applying the proper-layer assertion at $p$ yields the stated natural inclusion. At no point is $J(H)$ formed. □

The fixed-constructor order restricted to a proper layer is recovered by the same hierarchy records: old objects precede new objects, and new objects are compared by their first definitions. At a limit, the record for any particular initial segment fits in a proper layer with finite room. At a finite successor, its code table is defined over the last layer by the Def descriptions just constructed. This gives the order information used in the next lemma. It compares actual objects; the low code order has no mathematical role.

**Lemma 4.4** (Finite sets and overflowing tuples)**.** *The actual $<_L^{\mathrm{fin}}$ comparison of all finite-set trees over $L_t$ has a correct boundedly checked code table in $L_{t+1}$. This includes trees with arbitrarily large finite height above the last layer of $L_t$.*

*Proof.* A new finite set $X$ at a Def stage has an earliest direct constructor with $|X|$ distinct parameters. No shorter constructor defines it; the least list of that length is its increasing member list. All other formulas come later. Consequently two new finite sets at the same stage are ordered by cardinality and then by their increasing member lists. Different stages are ordered by stage.

Represent a tree over $L_t$ by a finite $A$-code. A value lying in $L_t$ uses the correct layer evaluation and layer order. A value outside $L_t$ lies at a positive finite band above the base. Unequal bands decide comparison. At equal bands the values are finite sets; deduplicate by actual equality and compare their member lists. Each member lies in a strictly lower band, so this recursion terminates. A finite trace of all these comparisons has only $A$-codes and natural numbers as fields and is itself in $A$. Its local checks are bounded. Conversely induction on bands makes a successful trace correct. Bounded separation over the code domain in $L_{t+1}$ gives the complete equality and order tables, with both directions of the checking equivalence. The objects compared are the original sets, not their low codes. □

**Theorem 4.5** (Uniform low canonical certificate)**.** *For the fixed order above, Gen and the canonical hull, cut and exact graph clauses of actual legal incoming maps at target $t$ have correct low representations at the required tight level $L_{t+1}$. The check is sound for newly quantified candidates as well as covering true old graphs. At a limit target the corresponding limit-level construction gives the same conclusion.*

*Proof.* Let $\Delta_t(i, \bar{a}, \bar{b})$ be the correct bounded-matrix check on codes of parameter and witness coordinates. Form, by the fixed ordinary tuple format, a tree code $\widehat{T}_i(\bar{b})$ for the complete actual witness tuple. By Lemma 4.4, its minimality is the bounded assertion

$$\Delta_t(i, \bar{a}, \bar{b}) \ \wedge \ \forall \bar{d} \in A \, (\Delta_t(i, \bar{a}, \bar{d}) \Rightarrow \neg O_t(\widehat{T}_i(\bar{d}), \widehat{T}_i(\bar{b}))), \tag{4.2}$$

where only valid codes of the prescribed arity are used. The code evaluation is onto the actual layer, so (4.2) quantifies over every actual witness tuple. Bounded separation gives the full selector table. Finite term computations give the full term table in the same level. From it, the codes of elements generated by the required seeds are exactly those with a finite correct computation.

For a candidate graph, verify the source and target structures, the membership isomorphism, the stipulated elementarity, the critical point, and equality of its image codes with those hull codes. Since the source is transitive, this image equality says precisely that the candidate is the inverse collapse of the specified hull. It also gives the cut and the required exact domain. Each of these checks is bidirectional, so a false new candidate cannot pass by supplying a spurious selection table.

For coverage of the actual incoming graphs used here, let $J : L_h \to L_t$ have critical point $p$ and $J(p) = c$. The actual field order gives $h < c < t$, so the source is a small structure below the receiving column. Its actual terminal source is an entry of column $p$, so it satisfies $\mathrm{Gen}_{\mathrm{fin}}(p, h)$.

Lemma 4.6, whose proof uses proper-layer elementarity and no low certificate for $J$, identifies the value of each source term with the same target term. Code a pair in the graph if it has these two evaluations of one finite term over corresponding generators. Every source element has such a term. Different descriptions of the same source element have the same target value because $J$ is an actual function preserving those computations. Thus the code covers the entire actual graph, not merely a subgraph. Conversely every pair so coded is a pair of $J$.

This construction uses bounded separation on the correct term tables; the graph code belongs to $L_{t+1}$. It does not assert that the whole ordinary function graph belongs there. The same selector and term tables directly check Gen. For an arbitrary newly quantified graph, totality and all the preceding semantic checks remain mandatory; source Gen is not a substitute for soundness of those checks.

At a limit target, each finite computation belongs to a proper layer with enough finite coding room. The union of their correct descriptions is definable over $L_t$ and hence has its code in $L_{t+1}$. This supplies the same selector, term and generated-graph checks. No unrestricted existence theorem for a weak hull is needed: the graphs whose coverage is used are the actual legal incoming graphs and the generated restrictions constructed in the proof. □

### 4.4 Elementarity and transport

Fix one formula convention throughout. The narrow convention is $\mathcal{E} = \exists\Delta_0$. The wider convention $B\Sigma_1$ is closure of bounded formulas under finite conjunction, disjunction, bounded quantification and existential quantification. At a weak endpoint these must not be silently identified by prenex conversion. For the narrow convention correct partial truth gives the graph check. For the wider convention use, for every formula code, a finite correct table for that formula and its subformulas. The syntax is "for each code there exists its finite correct table," not an unjustified assertion of a single total truth table in the weak level. Section 4.3 gives these two checks separately.

**Lemma 4.6** (Actual finite-term transport)**.** *An actual positive elementary map $J : L_H \to L_V$ preserves the fixed finite canonical computations, including at finite successor endpoints. If $p = \mathrm{crit}(J)$, $J(p) = c$, $p < g < H$, and $\mathrm{Gen}_{\mathrm{fin}}(p, g)$, then*

$$J''L_g = \mathrm{Hull}_{\mathrm{fin}\,J(g)}(p \cup \{c\}), \qquad J \restriction L_g \text{ is its exact inverse collapse.} \tag{4.3}$$

*Proof.* Proper layers and their relative truth are preserved by layer identification. At a successor endpoint, interpret the finite definition codes and their least descriptions over the last proper layer. Actual comparisons of overflowing finite constructors reduce to the finite tree comparisons of Lemma 4.4. Those comparisons have fixed first-order interpretations over the last proper layer, where the actual restriction has full elementarity. Thus the least description and its specified projection are preserved. The fixed-description argument is given in Section 4.5. Induction on finite term length then gives the assertion.

For (4.3), every element of $L_g$ is a finite canonical term from $p + 1$, by the expressly assumed source Gen. Its image is that term from $p \cup \{c\}$. Conversely each such target term is transported from the same source term, with definedness reflected by elementarity. This gives both inclusions and identifies the inverse collapse. Its source is exactly $L_g$. We use $g < H$ throughout; $J(H)$ is never formed. □

Two maps from the same actual canonical source that agree on its specified generators therefore agree on all finite terms. This is the map identity used along actual $\Gamma$ chains. It does not identify maps with different parent columns. When a whole package is transported, its new external incoming edge is the restriction in (4.3); the transported old incoming edge has a different role.

### 4.5 Why the fixed selectors commute with a weak map

We supply the least-witness argument in Lemma 4.6; in particular it applies to the entire actual map at a weak successor, not only to restrictions of a larger map.

If the source endpoint $H$ is a limit, the actual finite witness tuple $w$ belongs to $L_H$. Choose a proper $L_\beta$ containing the parameters, $w$ and the finite tuple construction. The canonical order is extended only at its end, so every candidate earlier than $w$ is already in that small layer. Its correct hierarchy, order and relevant truth records fit in $L_H$ after a finite increase of $\beta$. They give a positive certificate that $w$ satisfies the bounded matrix and that no earlier tuple does. In any transitive target that passes the same check, this is still the global first witness: everything outside the correctly identified initial layer comes later. Transporting this certificate through $J$ proves preservation of the witness and its projections. Definedness in the reverse direction follows directly by positive elementarity of the finite existential matrix. The target need not be admissible or a limit.

Now write $H = \lambda + n$ with $n > 0$. Preservation of the finite successor pattern gives $V = \mu + n$, where $\mu = J(\lambda)$. Put

$$M = L_{\lambda+n-1}, \qquad N = L_{\mu+n-1}, \qquad e = J \restriction M.$$

Lemma 4.3 makes $e : M \prec N$ fully elementary. The actual $J$ is its one Def extension: if $X = \{z \in M : M \models \varphi(z, \bar{a})\}$, then

$$J(X) = \{z \in N : N \models \varphi(z, e(\bar{a}))\}. \tag{4.4}$$

To see this, express the defining equivalence with all quantifiers bounded by $M$ and $X$, and use the proper-layer identity. There is no assumption that $J$ itself is fully elementary.

For each fixed finite layer, the true restriction of $<_L^{\mathrm{fin}}$ is first-order definable inside that layer with its proper lower layers as parameters. At the limit base use small-layer first-definition records. At a successor, satisfaction of any particular formula over the last proper layer is expressed by the existence of its correct finite subformula table in the current layer. This defines the relation formula-by-formula without a set containing the current layer's own full truth. First definitions are then compared using the previously defined order. The proper-layer parameters are preserved by $e$.

For an external fixed formula number $i$ and parameters $\bar{a} \in M$, let $X_i(\bar{a})$ be its defined subset of $M$. There is a finite first-order condition $\mathrm{First}_i(\bar{a})$ over $M$ saying that this is a new object and this description is its true first one. Newness excludes every old $u \in M$ with the same members. Minimality of the parameter tuple excludes earlier same-formula tuples. Earlier ordinary formula numbers give finitely many further exclusions.

The two-block formula order needs an additional check. If $i = \chi_s$, only the $s$ shorter constructors precede it. If $i$ is in the second block, all finite constructors precede it, so one must say that the defined set is infinite. This is first-order expressible over $M$: at a limit base there is no finite list exhausting its members; at a successor use a finite list of codes in $L_\lambda$, evaluated onto $M$. Every actual finite list has such a code. Negating exhaustibility therefore excludes the entire first block. Treating that block as a finite initial segment would be incorrect. Full elementarity of $e$ preserves $\mathrm{First}_i$ and hence the actual first-description order of objects in $\mathrm{Def}(M)$.

For a fixed finite tree of set constructors over $\mathrm{Def}(M)$, give each node still in that layer its true first description. Old nodes are parameters in $M$; new nodes have the descriptions above. The claim that a node is the finite set of its children is a fixed bounded condition, translated to $M$ using their definitions. Nodes outside $\mathrm{Def}(M)$ are assigned their actual finite bands over the limit base. Different bands decide comparison; equal positive bands are compared by the number of distinct members and their ordered member lists. Every comparison descends a band, ending at a described layer object. Deduplication and the chosen ordering of children are finitely many further

comparisons. Thus each particular finite tree comparison is equivalent to one finite first-order formula over $M$. No expression $J(T)$ is used for a tuple $T$ outside $\mathrm{dom}(J)$.

Finally fix a source query, its parameters $\bar{a}$ and its actual least witness coordinates $\bar{w}$. Their images satisfy the target bounded matrix. If a target tuple $\bar{v}$ were a strictly earlier witness, choose true descriptions of this particular tuple's coordinates and its finitely many intermediate tree nodes. Translate its bounded matrix and its strict comparison with the transported source tuple to one first-order formula over $N$. Quantify the candidate description parameters. All other parameters are $e$-images of the fixed source descriptions. Full elementarity reflects this formula to $M$. Interpreting the reflected descriptions gives an actual source witness strictly earlier than $\bar{w}$, a contradiction. All first-description, equality and order clauses were included in the formula, so this is a comparison of the original witness sets, not of their auxiliary codes.

Hence the selected coordinates, their projections and definedness are preserved. The argument works separately for every actual formula number and every finite arity; it needs no set of all description translations in the weak endpoint. Finite induction gives transport of all canonical terms and proves Lemma 4.6 without using Theorem 4.5. In that theorem, actual source Gen can therefore be used to code the complete true graph without a circular appeal to its own certificate.

# 5 Legal labels and the actual work models

## 5.1 The labelling predicate

A legal labelling assigns ordinals above $\omega$ to every actual column and entry of $C_{\mathrm{original}}(M)$, increasing in the stated depth-first field order. It assigns each parented entry $u$ in column $c$, with parent column $p$ and actual terminal consumer $G$, its canonical map

$$J_u : L_g \longrightarrow_{\mathcal{E}} L_t, \quad g = \ell(G), \quad t = \ell(u), \quad \mathrm{crit}(J_u) = \ell(p), \quad J_u(\ell(p)) = \ell(c).$$

The graph is the specified inverse hull collapse. Gen is required of every entry relative to its owner. A nonempty private height has the original D4 condition

$$L_{\ell(u)} \prec_{\mathcal{E}} L_{w(u)}, \tag{5.1}$$

where $w(u)$ is its last *direct* private-column label, not the deepest tip.

The fully completed scans determine the remaining conditions. For proper roots $r < g$, rootOnly requires $J_u(r) < s$ and the complete source package to be in the domain with all its images below the current target $s$; both requires $J_u(r) = s$ and equality of the entire corresponding private package; targetOnly requires $J_u(r)$ above $s$ and all its private fields. At a proper coverage opening, the root is the least ordinal whose image strictly exceeds the opening target. A persistent tag opened while more than one target remained requires every ordinal below $g$ to have image below that opening target. The corrected convention ignores the bookkeeping reset of a final both event in this persistence test. The exact domain $L_g$ is required at every actual terminal, including targetOnly. At the terminal itself one does not evaluate $J_u(g)$. Nonfinal resets still count. All these clauses are checked after completion and an independent rescan, on virtual as well as native entries.

The maximum of all labels is denoted $\rho(C)$; it is zero for an empty terminal table. The predicate just described is an external set-theoretic predicate on finite syntax, labels and actual graphs. The low certificates represent its particular internal instances; they do not redefine truth to mean merely that an unchecked object calls itself a certificate.

## 5.2 Strength of columns and admissibility of non-bare items

**Lemma 5.1.** *An actual critical map with critical column $p$ and receiving column $c$ gives the natural full elementary inclusion $L_p \prec L_c$. Both layers satisfy KP, full Separation and full Collection. Every actual nonempty direct private column has this strength. A non-bare item layer is admissible by its strict D4 relation.*

*Proof.* The layer-identification induction fixes every set below the critical column, not just every ordinal: $J \restriction L_p = \mathrm{id}$. Boundedly relativize any first-order formula to the correct layer object. Its transported interpretation is the corresponding relative formula at $L_c$. This yields natural full elementarity. See the layer-identification construction in Section 4.3.

If $L_p$ satisfies $\forall x \in A\, \exists y\, \varphi(x, y, u)$, the higher $L_c$ sees the set $L_p$ itself as a set containing witnesses for all $x \in A$. Full elementarity reflects the existence of a collecting set to $L_p$. For full Separation, first form the defined subset in $\mathrm{Def}(L_p) \subset L_c$, then reflect its existence. The elementary basic closures give KP, and each axiom instance transfers to $L_c$. No Power Set axiom is required.

An actual nonempty private column has a parented entry in its standard birth row. Completion retains that entry. Its actual critical map therefore proves the preceding assertion for that receiving column. Finally the strict natural D4 relation at a non-bare item gives its admissibility by the witness-collection argument below. This is not an admissibility requirement on every bare item. □

To justify the last assertion, suppose $L_t \prec_{\mathcal{E}} L_w$ is the natural D4 inclusion and $L_w$ has Collection. If $L_t \models \forall x \in A\, \exists y\, \delta(x, y, z)$ with $\delta$ bounded, then $L_w$ has witnesses already in the set $L_t$. Collection in $L_w$ gives a bounded collecting set; the assertion that such a set exists is positive and reflects to $L_t$. Bounded Separation and the elementary basic axioms hold in every relevant limit layer. Thus $L_t$ satisfies KP. The argument uses $t < w$ and the actual D4 inclusion, and does not apply to an arbitrary bare endpoint.

**Lemma 5.2** (Matching a strong work level)**.** *Every use of small-structure hull construction in the current argument has a work level with the required Collection strength and the actual elementary relations to its item levels.*

*Proof.* There are two uses. If the work model is the original $M = L_{w(R)}$ of a nonempty private height, retain that very model and the old relation $L_\kappa \prec_{\mathcal{E}} M$. Its strength follows from Lemma 5.1. It is not legitimate to replace $M$ by an arbitrary larger level and keep the old relation without proof.

For a common auxiliary level, the original supplying map $J : L_H \to L_V$ has critical column $a$ and $J(a) = c$. The finite low records $D_i \in L_a$ and open-item relations $L_{t_i} \prec_{\mathcal{E}} L_a$ have already been supplied. With $T = \mathrm{KP} + \mathrm{Separation} + \mathrm{Collection}$, reflect the single statement

$$\exists W\ [\beta < W < c\ \wedge\ \bigwedge_i D_i \in L_W\ \wedge\ \bigwedge_i L_{t_i} \prec_{\mathcal{E}} L_W\ \wedge\ L_W \models T]. \tag{5.2}$$

The high witness is $W = a$. Correct truth for the small structure $L_a$ is available in $L_c$, since the distinct critical and image columns are limits with enough finite room. The recursive axiom scheme is checked over formula codes in $\omega$. The parameters are fixed below $a$. The same $J$ gives a witness $W < a$, together with every displayed relation. The work-level endpoint certificate may remain in $L_a$ rather than $L_W$.

For completeness, the small-structure transitivity argument used in these models is the following. An element of a hull generated by $d+1$ has a finite single-valued positive definition from parameters of hereditary size at most $|d|$. In the strong work model take a small full elementary substructure of a sufficiently large set structure containing the witnesses and the parameters' transitive closure. Collapse it. Bounded absoluteness and uniqueness identify the collapsed output with the original

output; hence the output's hereditary size is at most $|d|$. An elementary hull containing $d+1$ then contains a surjection $d \twoheadrightarrow x$ for every nonempty $x$ in the hull, and contains every value of that function. It is transitive. Only after this argument is its collapse identified with a natural inclusion. □

### 5.3 Restricting a codomain and collecting an actual graph

If a positive map's image lies in a transitive sublevel $L_\rho$ of its target, it remains narrow positive elementary into $L_\rho$: source witnesses map into $L_\rho$, and witnesses in $L_\rho$ are upward absolute to the larger target. In the wider convention a bounded universal may require Collection. The actual protected stopping sources are non-bare, hence admissible. In such a source each $B\Sigma_1$ formula can be normalized by collecting bounded families of existential witnesses. The normalized existential formula implies the original formula in every transitive target. Transport its witness to $L_\rho$ for the forward direction; use upward absoluteness for the reverse. The target need not be admissible. This proves the restriction used at stopping points.

Whole function graphs are collected only in a verified admissible non-bare item layer $L_u$. The source is a small structure there and has its correct finite-term table there. The target evaluation in the limit $L_u$ has a correct single-valued positive definition. Lemma 4.6 gives totality and uniqueness. Then $\Sigma_1$ Collection and $\Delta_1$ Separation put the entire graph in $L_u$ and give its ordinal-image supremum strictly below $u$. This is a separate operation from low coding at a weak successor layer.

**Corollary 5.3** (The certificate is in the actual reflection level)**.** *Let $D$ be a true finite record used in an actual reflection in $L_V$, and suppose every internal graph target $t_i$ is strictly below $V$. Its complete correct check is available in $L_V$, including the case $t_i + 1 = V$. A new candidate satisfying the reflected check is legal for the same fixed canonical system.*

*Proof.* Theorem 4.5 places each internal graph's data in $L_{t_i+1} \subseteq L_V$. Finite fields are represented by low codes or separately quantified objects, so no boundary pairing closure is assumed. Scan, parent, exact-domain and package correspondence conditions are bounded checks on the correct data. D4 and open-item stability use the matching elementarity convention. Low-side closed certificates are reconstructed below the receiving column, whose strength was proved above. The supplier $J$ itself and its ambient domain endpoint are not internal fields of $D$. For the wide convention retain the per-formula finite-table quantifiers. Coverage and soundness follow component by component. □

## 6 Common Point supply and installation

### 6.1 What a Point asserts

A Point for a finite standard target $P$ at owner $a$ consists of an entry label $\rho$, its fully completed private table $T$, actual internal incoming maps, correct certificates, and the necessary external support $\Delta$. It asserts the correct native projection, whole-package identities, scope, order, Gen, D4 when nonempty, and all actual scan conditions. The external incoming edge of the new entry is supplied at installation, from its actual consumer; it is not confused with an incoming edge of the source package.

An intrinsic Point asserts these internal conditions before that new external edge has been installed. The return value contains the actual source occurrences and support maps, not just a drawing of a private table. If a call enters a more external containing frame, all newly internal support, labels and certificates are quantified together again. A previous witness can prove existence of a high configuration; its arbitrary chosen values are not new fixed input requirements.

The admissible working environment consists of the current legal source, already supplied low connection records, closed proper packages with their real graphs, the finite set of old open-terminal prefix constraints, actual lower bounds, and the operation's permission. The supply theorem does not confer that permission.

### 6.2 Sharing a source and stopping at a real bound

For a proper strict coverage opened at root $g$ and target $u$, the least-overshoot condition gives

$$g = \min\{\alpha < \gamma : J(\alpha) > u\}.$$

Both entries are non-bare. Since $g$ is a limit, for $\alpha < g$, $J(\alpha) < J(\alpha + 1) \leq u$. The actual $\Gamma$ chain from the target back to that root composes to $J \restriction L_g$, by the same finite source descriptions. Its final non-bare map has its graph internally collected at $L_u$, so

$$\sup J[g] < u. \tag{6.1}$$

For finitely many readers of the same actual source, all these images are bounded by the same final actual map. Choose the stopping label above that bound and the genuine finite predecessor bounds. No claim of cofinality for every intermediate image is needed. This supplies shared lower bounds without making different choices for one source. The same restriction identity is used for all occurrences of one actual source; different parent columns are not identified.

### 6.3 Synchronizing a complete source prefix

Fix an actual old map $J : L_g \to L_t$, or an actual connection map already present in a low record. In addition to the two terminal positions, retain the full closed proper prefixes, their actual consumers, and the restriction identities of their graphs. The new private height at the terminal may still be open. No clause here asserts that this new terminal is already a legal completed entry.

**Lemma 6.1** (First-gap synchronization)**.** *Given a correctly installed left Point and the finite closed source prefix that must precede it, one can synchronize that prefix along all its actual old readers and return the Point as one whole package. The construction preserves their old complete-prefix conditions. Source roots are paired through actual both chains; old target coverage blocks may remain between the paired targets.*

*Proof.* Find the first source root in the required prefix not yet paired. If it was removed by rootOnly in the next request, follow that request's actual consumer chain to the source that supplies its whole closed package. Along an ancestor route, G places the consumer before the containing head, so the package is closed. Along a connection route, use that connection's recorded map and continue back to its Init record. Transport the whole source package forward along these same maps, and insert each image before its actual right carrier. The new external incoming edge at a label $J(\sigma)$ is $J \restriction L_\sigma$; its domain is exactly $L_\sigma$ by Lemma 4.6. An image of a source head's old incoming edge is an internal record with a different role, and is not used as this new edge.

For an old proper reader, finite-term identity on its actual source identifies the restriction used in the transport with the reader's restriction. Thus an old both event remains a full package equality. An added rootOnly package lies below its unchanged right carrier and inherits that carrier's whole-image bound. An added targetOnly package lies below its actual closed target carrier and uses the strict or whole-domain bound of that old coverage block. At a terminal root, domain membership and the codomain bound replace the nonexistent value $J(g)$. The persistence condition uses the old opening target and ignores only a final terminating both reset. These are precisely the three scan events, with their domain obligations.

For several readers of one source, choose the same package once and combine their finite lower bounds and the common stopping bound (6.1). Use its corresponding image along each real route. Source-prefix synchronization may transport several intermediate packages, but each transport fills the first missing proper pair in a finite fixed source prefix. Backward travel goes to an earlier actual column or an earlier connection. Thus it terminates.

After synchronization, let $G$ be the next source root and let $s$ be its first target consumer. Every earlier required source root has already been consumed by its paired target before $s$. If the new height $P$ is legal at the source and $P < \mathrm{ht}(G)$, then $P < \mathrm{ht}(s)$: otherwise candidate minimality at the first consumer would put $C(\mathrm{ht}(s)) \le P$ below $G$, so $G$ could not begin that block. Hence a Point before $G$, above the entire required preceding prefix, returns to a package before $s$. Repeat at the first actual consumer of $G$ in every intermediate column. This locates the insertion; the final equal-height target of $G$ need not be its first consumer.

The prefix premise is essential. Without it an earlier unpaired source root could consume the new target before $G$ was reached. The preceding first-gap procedure supplies exactly that premise. Replaying and pruning the closed packages preserves physical identity by Lemma 2.5. □

**Lemma 6.2** (Necessary support is smaller)**.** *Let $P$ be an actual standard target at owner $q$ and suppose $C_r(P) = P$. Every support head $B$ actually consumed by an internal reader of $P$, at an external visible column $b \ge r$, satisfies*

$$C_r(B) = B, \qquad d(B) < d(P), \qquad \nu(B) < \nu(P). \tag{6.2}$$

*The assertion includes old support that remains necessary, but not an unpruned larger seed merely transported temporarily into the framework.*

*Proof.* Because $P$ is exactly legal at $r$, its true external parents are all below $r$. The first exit from its private region to a visible column $b \ge r$ must therefore be through its owner $q$. Follow a particular consumed root back to the first target $Y$ it consumes. Either the first event is both, so $B = Y$, or it opens coverage, so $B = C_q(Y)$. If the same root continues to cover later targets, retain this original opening $Y$ as its source.

The standard template of $Y$ is a proper private subheight of $P$ or an internal candidate descended from one. G bounds its depth strictly below $d(P)$, and proper containment bounds its size below $\nu(P)$. Cut does not increase size and preserves a retained nonempty first branch. Fields outside $Y$ but still inside $P$ are cut at the fixed external owner; any true external field left in $B$ comes from $P$ itself and is below $r$. Hence $C_r(B) = B$. The same proof applies to the internal requests of this support package, with strictly smaller size at each exit. Birth-record induction proves (6.2).

If a larger old package has been transported, first prune it to the support actually used by the retained readers. A retained head is analyzed through its first actual consumption as above. Its old entire source prefix is an installation obligation, not a new internal target of size $\nu(B)$; Lemma 6.1 handles that prefix. □

**Lemma 6.3** (The next exact support cannot be skipped)**.** *Let $D_1, \dots, D_k$ be the union, in actual position order, of the necessary roots used by finitely many internal readers of an exactly legal $P$. Deduplicate actual occurrences, not equal heights. After $D_1, \dots, D_{i-1}$ have been paired through the outer comparison, the request at $D_i$ either uses its equal actual root or requires the exact insertion $D_i$. It cannot persist under a larger root.*

*Proof.* Every intervening target $E$ between the last paired root and $D_i$ has $E < D_i$. By (6.2), $D_i$ is legal in the outer root class, so $C_r(E) \le D_i$. Any coverage opened in this interval therefore has a root at most $D_i$. On arriving at $D_i$, a smaller current root is removed by rootOnly and its tag is reset; an equal root gives both. A strictly larger first root has no persistent tag, so the minimum candidate rule requests $D_i$ itself. Root exhaustion gives the same exact request. This proves the assertion by induction on $i$.

Composition of qualified scans shows that each resulting proper pair has an actual both chain through all intermediate columns. Merging the necessary roots of several readers changes neither this chain nor their inner consumers: omitted roots were unused, and retained required roots stay in their actual order. Larger temporary shadows are not added to the necessary word merely because they appeared during preparation. □

To apply these lemmas during an open outer call, carry all its actual old terminal-prefix constraints from the beginning, including every old graph along a possible backward route and the connection route to Init. A local insertion before the open terminal has a closed proper right carrier. A root-side insertion uses that carrier's whole-image bound. A target-side insertion with proper source uses its actual restriction and the coverage bound; one with terminal source uses the whole-domain version. Several readers use one shared source. These checks do not read the new open private height. A constraint on that entire future height is left to its outer return and is not prematurely declared satisfied. External old graphs need not themselves be low parameters of the local existence formula: their restrictions are checked on installation, and their local numerical demands enter only as the proved common finite lower bound.

### 6.4 Reflecting a jointly chosen source

The outside support of a Point has two kinds of data. A package in a strictly earlier external column has its closed fields and correct certificate below that receiving column. A package on the current owner before the terminal has its fields and certificate below that terminal. The latter may be above the critical point. It need not be fixed by the current map; it must have an actual source correspondence.

**Proposition 6.4** (Common-source reflection)**.** *Let $J : L_g \to_{\mathcal{E}} L_t$ be an actual recorded map, with critical point $a$ and $J(a) = b$. Suppose $C_a(P) = P$. Assume:*

1. *the necessary finite source support $\Sigma$ is already correctly installed before $g$, using smaller calls and the old prefix constraints;*
2. *synchronization gives the target intrinsic support $\Sigma' = J(\Sigma)$ along actual both chains; each side's external incoming edges and installation certificates are checked separately;*
3. *the closed internal construction of $P$, together with this support, gives a true intrinsic high Point and hence Points cofinally below $t$ by the old natural work-level inclusion.*

*Then for every $\beta < g$, above the required source-prefix bound, there is an intrinsic $P$ Point with label $\rho$ satisfying $\beta < \rho < g$. Its installation and return use the same chosen support.*

*Proof.* Use $J(\beta) < t$ as the target lower bound in the third premise. The existential formula describing the entire intrinsic Point has parameters $J(\Sigma)$ and the images of the genuine fixed external ports. Its internal fields, graphs and certificates are all quantified. The correct low check belongs to the actual $L_t$ by the certificate corollary. Positive elementarity of $J$ reflects this very formula to $L_g$, with parameters $\Sigma$ and bound $\beta$. It produces a source witness, not an inverse image of an arbitrarily selected target table. Lemma 6.1 installs and returns it with external edge $J \restriction L_\rho$.

The source support may lie above $a$: only its actual image equation is used. Nor must an old target covered by a proper root equal the image of that root's endpoint. Its actual incoming map agrees with the restriction below that endpoint, which is all that transporting the newly chosen $\rho$ requires. On entering a more external existence statement, $\Sigma$, both internal tables, all graphs and their certificates are quantified together again. No arbitrary earlier choice becomes an extra eligibility condition on the outer call. □

The third premise is obtained as follows. Let the actual old outside item have label $t$ and retained relation $L_t \prec_{\mathcal{E}} L_W$. After all smaller calls have returned, its internal target is closed, with actual last direct column $q$ satisfying $t < q < W$ when nonempty. Use the old label $t$ as a logical high witness. Its Gen is unchanged; D4 restricts to the actual $q$; all private graphs and correct certificates are present. Reflect the existence of this whole finite configuration, with all its variable fields, along the natural inclusion to $L_t$. For any $\eta < t$ it gives a Point with outside label between $\eta$ and $t$. For an empty height omit D4. There is no premise that $P$ is a prefix of the old height.

The first two premises are supplied by the smaller calls themselves. Equation (6.2) makes every necessary support a strictly smaller target. The old terminal-prefix constraints accompany those calls. On return, Lemma 6.3 gives actual proper both sources before $g$; their intrinsic records and source certificates lie in $L_g$, and their target records are the actual $J$-images. Thus no second task is created to find an inverse image of an already chosen support. This is the closure needed for the simultaneous induction below.

## 6.5 The simultaneous induction

**Theorem 6.5** (Common Point)**.** *In the qualified working environment, actual active targets and recursive-prefix targets admit simultaneous Point supply and installation, with necessary non-bare owner support, all retained readers, and all open-terminal prefix constraints preserved.*

*Proof.* Use strong induction on the finite size $\nu(P)$ of the standard target. The internal target of a nontrivial call is a proper private subheight, so its size is smaller. The assertion includes the full return value and installation; assuming only an uninstalled internal diagram would be insufficient.

Empty heights and deletion of whole tails use the whole-entry downward statement of the old non-bare carrier, or retain an existing bare correspondence. Recursive-prefix calls use the same statement after constructing their last retained internal target. A genuinely new internal target is smaller, so the induction supplies its table, support, actual maps and certificates. Old closed packages are reused. Every necessary support template is also smaller than $P$; at its certified query it is exactly legal and has lower first- branch depth. Temporary larger shadows are traced to their actual birth sources rather than used as new induction targets.

For a backward main query use the complete classification: parentless carrier, equal height, unchanged candidate, old persistent coverage, renewed persistent shadow, or an exact strict cut. An unchanged candidate retains the old *actual* consumer even if there was no separately present minimal-candidate head. Persistent coverage stops at its actual carrier. A strict barrier is a prefix carrier by Section 3 and uses prefix supply. An exact nonprefix query continues along the actual old $\Gamma$ route, or through a real low connection until its Init record. This route uses earlier actual columns or earlier connection copies.

The crucial simultaneous invariant is the old graph's complete constraint on its terminal prefix, even while the terminal's new private height is open. If an insertion is only on the root side, its whole package is below its right carrier, and the old proper image bound (or terminal codomain bound) pays the new rootOnly events. If an insertion is on the target side with a proper source consumer, its actual $\Gamma$ chain identifies the old map's restriction; equality transports one whole source package, while coverage uses (6.1). If the consumer is the terminal root, use the whole-domain version of the coverage bound, never the undefined value of the domain endpoint. Multiple readers use the same source and the common stopping bound. These checks refer only to closed proper packages and the real old maps, not to the terminal's unfinished new height. This proves the invariant. The installation argument in Section 6.3 explains the source-prefix synchronization used here.

Consequently when smaller calls return the necessary owner support, that support already has actual proper both sources in the larger old graph. Its source certificates lie in the domain and its target data are the genuine images. One does not now ask for the inverse image of an arbitrarily

preselected support. Carry the entire finite old route and real connections from the start; a later move to an earlier strict carrier becomes prefix supply. The internal table can therefore be assembled into a true intrinsic high Point. Keep the old outside entry label and its Gen; restrict D4 to the new last direct private column. Correct certificates exist by Section 5. Reflect the whole existence statement with the actual finite lower bound through the old natural inclusion and, when needed, the actual old graph. This supplies the source Point above that bound. It does not require the target to be a prefix of the old height. The precise common-source reflection is Proposition 6.4.

On return, transport a source entry and its complete private package together. Lemma 4.6 supplies its new incoming restriction and exact domain. Internal old edges are transported in their own roles. Proper readers are identified on the same finite source terms; coverage uses the internal graph bounds and the justified codomain restriction. Retain the old terminal-prefix constraints for all containing open calls. Freeze closed packages, replay their creation records and prune to the least actual support. Lemma 2.5 then identifies the result with the original completed target.

There is no circular termination argument. Smaller target calls decrease $\nu$; same-size main travel decreases actual column or connection-copy position; synchronization in a fixed prefix pays successive finite gaps. None of these measures is future native expansion. Internal witnesses are jointly reselected on entering an outer frame. This completes the induction. The smaller-support and synchronization lemmas above account for all auxiliary calls, including non-bare owner prefixes. □

# 7 Every native step lowers an arbitrary legal source

## 7.1 Preparation and one-map supply

For a general native step freeze the original completed last request. Write its supplier as

$$J_* : L_h \longrightarrow_{\mathcal{E}} L_v, \qquad \mathrm{crit}(J_*) = a, \qquad J_*(a) = c.$$

Stage 0 is permitted only if this frozen request contains rootOnly. It transports original closed packages, and their continuous whole images, into the required proper gaps. If a root lies in an ancestor column, Theorem 3.1 places its actual consumer before the containing entry. Thus every source package ends before the original tail column $c$. Each successive transport has a source below that same fixed column. It never first raises an internal piece and then raises its already modified enclosing package. Every operation pays a gap in the finite original proper-root prefix, which does not grow. This proves finite preparation and its bound.

Stage 1 uses only $J_*$. Let $P$ be the prepared root-prefix boundary and $Q$ the tail-prefix boundary. The record $S(X)$ contains the closed side packages, actual old graphs and consumers, and the old data of open entries. Side fields above $a$ are quantified, not frozen. A connection $R(X, Y)$ additionally contains every direct incoming edge of $Y$, its private conditions and the open-entry stability relations to its right column. Init records the first boundary's actual incoming edges. Terminal targetOnly cases retain their exact domain equations. No terminal package crossing $h$ is wrongly put into the domain of $J_*$.

The prepared source gives true high witnesses for $R(P, Q)$ and $\mathrm{Init}(P)$. Closed low-side certificates lie in $L_c$, and right-side certificates lie in the actual $L_v$, including tight successor cases. Reflect the finite connection statement along $J_*$ to insert the first low boundary. To insert the next, keep each already supplied low record and its actual relation $R(A_i, P)$; the high witness uses $P$ as the next boundary and $Q$ on its right. Repeat finitely many times, then apply the final cap statement to put the required closed records in $L_a$. Retaining $R(A_i, P)$ also preserves stability of every open item to $L_a$. Apply Lemma 5.2 to obtain a common $W < a$. Now discard the original high supplier and high data. The remaining real low connections and Init carry all later uses. The finite connection formulas are written out below; their certificates are supplied by Theorem 4.5.

### 7.2 The finite connection formulas

Here $P$ and $Q$ name only the finite paired boundary fields, all source fields being in $\mathrm{dom}(J_*)$, so $J_*(P) = Q$. The source endpoint $h$ is not one of those fields. The side record $S(P)$ may extend beyond $L_h$; no assertion $J_*(S(P)) = S(Q)$ is needed. Its closed fields and certificates are in $L_c$. A side field can lie between two boundary item fields; the required inequalities are

$$c(X) < \text{variable side field} < c(Y) \quad \text{in } R(X, Y), \tag{7.1}$$

together with the actual complete field order. Fixed earlier ports are not variable side fields. Open old items retain their old labels, Gen, actual graphs, exact domains and work-level relations; their new private heights are not yet asserted to be complete.

**Lemma 7.1** (One-supplier finite connection)**.** *Suppose the prepared source supplies true $R(P, Q)$ and $\mathrm{Init}(P)$, correct low-side certificates in $L_c$, and no omitted external requests. For any finite number of copies and any $\beta < a$, the original $J_*$ alone supplies an ordered chain of low boundaries, all its required side records and incoming edges, and the final low certificates. All new fields are below $a$; the final closed certificates belong to $L_a$.*

*Proof.* The high witness $X = P$ gives in $L_v$

$$\exists X\, [\beta < c(X) < c \ \wedge \mathrm{Init}(X) \wedge R(X, Q)]. \tag{7.2}$$

Reflect through $J_*$, whose fixed external parameters lie below $a$. It gives $A_0$ below $a$, with $\mathrm{Init}(A_0)$ and $R(A_0, P)$. Equation (7.1) puts all its side fields below $a$, even if they interleave the item fields of $A_0$.

Suppose the last low boundary is $A$ and $R(A, P)$ has already been supplied. Its source certificate lies in $L_h$, hence also in $L_v$, where its bounded check remains correct. Use $Y = P$ as the high witness for

$$\exists Y\, [c(A) < c(Y) < c \ \wedge R(A, Y) \wedge R(Y, Q)]. \tag{7.3}$$

Keep the previously required low fields and earlier connections in the formula. They lie below $a$ and are fixed by $J_*$. Certificates not yet below $a$ remain existential fields; they are not falsely treated as fixed parameters. Reflection gives $A < B < a$ with $R(A, B)$ and $R(B, P)$. Retain all needed relations $R(A_i, P)$, as well as the consecutive connections. Their real records preserve each open item's stability to $L_a$. Repeat this finite step as many times as the specified native expansion requires.

Finally use $Z = P$ and the actual $S(P)$ to cap the entire low chain. In the high existence statement require every final closed certificate separately to belong to $L_c$. This statement has a true witness: the earlier connection certificates are in $L_h \subset L_c$, and the original closed side certificates are in $L_c$ by hypothesis. Since $J_*(L_a) = L_c$, reflection gives the final cap below $a$ and all the required certificates in $L_a$. Retained incoming graphs are part of the resulting data; none has been used as an additional stage-1 supplier.

Choose the common work model with the same $J_*$, by reflecting the joint statement (5.2), which includes both its strength and the actual open-item elementary relations. Then discard $P, Q, h, v, J_*$ and the high records. Future backward travel uses only the actual low connections and their Init source. The induction has taken finitely many downward uses of one original map and has not used the output of a previous native step as an extra assumption. □

The preparation supplies the hypotheses of this lemma, not merely an isomorphism of displayed heights. Proper rootOnly gaps are filled by whole images along actual old routes, and complete both packages are kept. Old coverage blocks remain as side records. Every direct incoming edge at a right boundary belongs to $R$, including targetOnly terminal edges with their exact domains. Lower-side closed graphs have targets below $c$, so the tight certificate theorem places their data

in $L_c$. Right-side graphs have targets below $v$, so their data are in the actual $L_v$. A certificate for a relation to the high boundary $P$ may remain in $L_h$ until the cap. These are different layer bounds; placing every intermediate certificate in $L_c$ without this argument would lose the tight bare-endpoint case.

Stage 0, when permitted, is a finite source-prefix synchronization of the original last request. Freeze that request before adding anything. At its first missing proper pair, the actual consumer route reaches a complete package strictly before the original last column. If the root is an ancestor, the strict-before-containing-head conclusion of Theorem 3.1 supplies this bound. Transport that entire old package; subsequent transports use its continuous whole images, each still before the same original last column. The next unpaid original proper-root gap moves forward. No internal modification is performed before transporting its already changed enclosing package. When the frozen request has no rootOnly, none of these stage-0 operations is used; its existing qualified coverage and both skeleton provide the appropriate records directly.

### 7.3 Permission before Point construction

The stage-2 permission concerns all adjacent native truncations $M[n+1] \to M[n]$, on their actual completed tables. If they all preserve every retained complete $\Gamma$, stage 2 must create no new labels, including temporary Point labels.

There is a finite syntactic dichotomy. Remove low-depth letters from a scan: they cannot consume a high-depth target, cannot first open coverage at a high root because their minimal candidate still has low depth, and cannot remove a high root without a high target. Thus the high-depth consumer skeleton remains. For a positive active result, either an exact active request has no old sustained coverage, or all active requests have candidate retention or old coverage. In the first case the finitely many old and preparation templates have bounded size, whereas the positive copy targets grow unboundedly. Choose $N$ beyond that bound. The strict-barrier classification excludes an intermediate prepared carrier, so the actual completions at $N, N+1$ have different exact consumer templates. This is a concrete adjacent change witness. In the second case the finite scans preserve the actual old consumers for all positive indices. Finally compute and replay the actual zero-index completion separately. A change gives the 0/1 witness; no change gives universal preservation. The zero case cannot use an unpruned frozen prefix in place of its actual $\Gamma$. This proves the permission dichotomy before any stage-2 Point is used.

**Theorem 7.2** (Arbitrary-source descent)**.** *Let $M \in \mathscr{U}$ be nonterminal and let $C$ be any current legal labelling for the fixed canonical system. For every $n < \omega$ there is a legal labelling $C'$ of $C_{\mathrm{original}}(M[n])$ with*

$$\rho(C') < \rho(C). \tag{7.4}$$

*No extra condition says that $C$ must have been selected by a particular preceding step.*

*Proof.* In the universal-preservation general case, stage 1 has supplied the closed branches, arrival blocks, old consumer packages, actual connections and old open labels. Return from the inner active call outward using only those fields, retaining the open label, Gen and old map. Restrict D4 to the new direct last column or remove it when the height becomes empty. Every required virtual support has a retained reader and an actual birth record; induction on those finite records shows it is already among the supplied packages. Unsupported extra shadows are pruned. Hence no Point and no new label is created in stage 2. Replay proves physical identity with the target completion.

In the other general case the adjacent change witness already establishes permission. Stage 1 supplies the working environment; Section 3, including Lemma 3.7, supplies the candidate menu and strict-barrier hypotheses. Apply Theorem 6.5 to the inside-out single open chain, carrying the outer old-terminal prefix constraints from the beginning. Each return is installed with its actual

incoming restriction, and complete old readers are synchronized. Outer existence statements reselect internal witnesses jointly. Original replay and minimal support identify the final table, not merely its native projection. All new windows are below the original general root, and the original high terminal is removed. The maximum label therefore strictly decreases.

For a special step, do not invoke the general stages. Compute its actual retained old $\Gamma$ changes. Inside the last changed old retained entry $t$, the unchanged parts use pure whole-entry downward construction; their old coverage or candidate retention keeps the exact incoming domains. If no retained old consumer changes, the whole step has no upward operation. Otherwise the first changed entry encountered from inside out is the last changed entry in physical order. Its internal candidate is now closed. Apply Point supply from the actual sources before $t$, not by beginning a fresh upward operation inside the old private table of $t$. Every actual upward field is strictly below the old label of $t$. Any deeper temporary witness is quantified anew in the whole existence statement. Treat the finitely repeated special copies in their actual order, then replay and prune. Deletion of the old deepest tail and the strict old bounds give (7.4). Section 7.4 records the separate bound and the inside-out return.

An empty-last-column successor simply deletes that last column and retains the earlier completed requests. It too strictly lowers the maximum. These are all native branches. Sections 4 and 5 provide the model-theoretic facts used in these constructions. In particular the uniform low certificate is used here, not replaced by an assumption about future Point existence. □

### 7.4 The special return and its old bound

For a special step let $R$ be the actual retained old occurrences, and let $\pi$ be the retention correspondence into the completed target. Define

$$D = \{u \in R : \Gamma_M(u) \neq \Gamma_{M[n]}(\pi(u))\}. \tag{7.5}$$

Both consumers are computed on their own original completed tables; the comparison uses actual occurrences under retention, not new program identifiers. If $D \neq \varnothing$, let $u_*$ be its last member in the original depth-first item order and put $b = \ell_M(u_*)$. The permission for this special operation is bounded by this old $b$. The construction above obtains every actual upward field strictly below $b$, which is stronger than the allowed weak bound.

To see why this bound is available, traverse the unique changed containing chain from the inside outward. Before reaching the first member of $D$, candidate retention or old persistent coverage keeps each actual incoming consumer. Complete-tail deletion and recursive prefix supply there use the old whole-entry downward statement: the outer label and Gen can remain, its old graph remains exact, and D4 restricts to the new last direct column. Any newly required low support is read from the retained complete packages and certified by their birth records. No upward operation has yet been used.

The first changed retained entry encountered is $u_*$. All genuinely smaller internal targets at that call are now closed. Thus the old entry gives a logical high Point using its retained D4 work model. Apply the common-source existence statement to actual sources before $u_*$, choosing the entire new Point below its old label. This uses the closed high configuration as an existence witness; it does not first raise a fresh object inside the old private height and then raise the enclosing package. Any inner auxiliary values used to exhibit that configuration are quantified anew with the whole Point.

Continue the outside returns with the shared source bounds, preserving all old complete-prefix readers. If the special rule makes several same-parent copies, finish them in their actual order; a future copy whose private height is still open is not a closed proper right carrier. The larger outside old bounds are unnecessary: every new upward field still lies below $b$. If $D$ is empty, this changed-entry step is absent and the entire special construction has no upward operation. Finally

replay and minimal support identify the target with its original completion. The deepest old tail is deleted and no field reaches the old maximum, proving strict maximum descent.

This separate argument uses neither the general connection stages nor the adjacent-index permission criterion. A change of a retained old height by itself is not a witness for (7.5); an actual change of its complete consumer is required.

# 8 Uniform initialization below the third Silver

## 8.1 Two fixed generated levels

We use the usual consequence of $0^{\#}$: a shift of Silver indiscernibles extends to an elementary $j : L \to L$. Put

$$a = \iota_0 < b = \iota_1 < \theta = \iota_2, \quad \mathrm{crit}(j) = a, \quad j(a) = b, \quad j(b) = \theta.$$

The indiscernibles are inaccessible in $L$. Form the fixed canonical positive hull $H = \mathrm{Hull}_{\mathrm{fin}b}(a+1)$. The hereditary-size argument in Lemma 5.2, carried out in the strong $L_b$, puts it in $L_{(a^+)^L}$ and gives $|H|^L = a$. For each nonempty $x \in H$, elementarity supplies a surjection $a \twoheadrightarrow x$ in $H$; its values are in $H$, so $H$ is transitive. Condensation gives $H = L_g$ with $a < g < (a^+)^L$. Compatibility gives $\mathrm{Gen}_{\mathrm{fin}}(a, g)$. The natural positive inclusions through $L_b$ give $L_g \prec_{\Sigma_1} L$; in particular $L_g$ is admissible by the witness-collection argument. Set $g' = j(g)$. Then

$$a < g < (a^+)^L < b < g' < (b^+)^L < \theta, \quad L_g \prec_{\Sigma_1} L_{g'}, \quad L_g, L_{g'} \models \mathrm{KP}. \tag{8.1}$$

All objects here are sets except the background shift; each restriction needed below is reconstructed as a canonical small hull graph.

## 8.2 Finite tower prefixes are reselected together

In an admissible work level, small layer structures, their satisfaction relations, canonical tables, finite terms and collapses can be constructed by $\Sigma_1$ Collection and $\Delta_1$ Separation. Hence a fixed finite labelling pattern with all endpoints below that level has a correct existential bounded certificate. This includes the statement that each chosen initial item layer is admissible. Only small structures' satisfaction is used; not the work level's own truth set.

**Lemma 8.1.** *For every finite $m \geq 1$ and $r \geq 0$ there is in $L_g$ a correctly labelled root prefix $Q_{r,m}$ consisting of $m$ bare entries followed by $r$ non-bare heads. Each new head's private column is a complete intrinsic image of the preceding root prefix. All initial item layers can be chosen admissible, and every initial request has its exact canonical domain.*

*Proof.* For one bare entry, $t = g$ is a high witness in $L_{g'}$ to $\mathrm{Gen}(a, t)$ and admissibility, with $t$ beyond any specified $\beta < g$. Reflect the entire certified existence statement along the natural inclusion in (8.1). For $m$ bare entries use a previously existing finite prefix and the final high witness $g$, but re-quantify all labels and certificates of the new $m$-entry record. This proves the base $r = 0$ by finite induction; it does not require a prescribed infinite auxiliary sequence.

Assume a whole $Q_{r-1,m}$ exists in $L_g$. Use it as a high witness prefix, put a new head at label $g$, and place $j(Q_{r-1,m})$ in its private column at $b$. For each direct source entry $t < g$, its new incoming map is $j \restriction L_t$. By its source Gen and finite term transport,

$$j''L_t = \mathrm{Hull}_{\mathrm{fin}j(t)}(a \cup \{b\}),$$

so this is the exact canonical restriction. Internal incoming maps move with their own packages. The new outside head has $\mathrm{Gen}(a, g)$; D4 is $L_g \prec_{\Sigma_1} L_b$. All proper comparisons are both.

Reconstruct the canonical maps and their correct small certificates inside the admissible $L_{g'}$. Reflect the single assertion of the *whole* enlarged prefix to $L_g$. The earlier labels are quantified again, with only the owner, pattern and finite lower bound fixed. This proves the induction on $r$. The finite tower templates are specified immediately below. □

### 8.3 The exact completed tower template

In one-based display notation define

$$H_{1,m}(q) = (\underbrace{q, \dots, q}_{m}), \qquad H_{r+1,m}(q) = (q^{H_{r,m}(q+1)}), \qquad V_{r,m} = ()(1^{H_{r,m}(2)}).$$

Thus $V_{r,1} = I_{r+1}$. The completed root column has $m$ bare entries followed by heads $G_1, \dots, G_r$. The private column of $G_j$ is a full intrinsic image of the root prefix consisting of the $m$ bare entries and $G_1, \dots, G_{j-1}$. The outer target column is a full image of the whole root prefix. Every request pairs both entry-by-entry, including its recursive private packages.

This is the original completion, not an added saturated table. Induct on $j$. A request in the $j$th private package needs exactly the equal-height same-parent roots in the earlier root prefix; they are already present by the induction hypothesis. No strict candidate is needed. Conversely each listed support has an actual reader in that private package or in the outer image. All internal packages have the same property recursively. Hence the table is completed and has the least actual support. Lemma 2.5 identifies it with the original completion of $V_{r,m}$.

Only the outer image of the last head is native at its level. Down its private chain the last head remains native, ending in the $m$ native bare entries. Other heads are virtual; source packages are globally virtual but retain their nonempty relative birth templates. This distinction explains why removing all globally virtual packages would destroy the initialization. For reference, the completed finite shape has $2^{r+1}$ columns and $2^{r+1}(m+1) - 2$ entries, obtained by doubling the preceding prefix and adding the next head. These are shape counts only; the ordinal labels are supplied by the joint reflection in Lemma 8.1.

**Theorem 8.2** (Uniform initial supply)**.** *The fixed $g' < \iota_2$ in (8.1) satisfies*

$$\forall k < \omega \; \exists C_k \, [C_k \text{ legal on } C_{\text{original}}(I_k) \;\wedge\; \rho(C_k) < g']. \tag{8.2}$$

*Proof.* Take a whole prefix of Lemma 8.1 in $L_g$ and its outer image at column $b$. Reconstruct the new direct incoming edges as the restrictions just proved. The complete finite template has only both comparisons; the first seven conditions and D4 hold recursively. Its labels and all certificates lie below $g'$. For $k \geq 2$ use the tower template $V_{k-1,1}$; $k = 1$ uses $Q_{0,1}$ and its image, and $I_0$ only needs an empty root column. Thus the same bound works for every finite tower. Initial admissibility is a choice in this existence proof, not a new premise of Theorem 7.2. □

## 9 From finite paths to the original well-order

### 9.1 A finite reachability theorem independent of well-ordering

Write $P \preceq_{\text{pref}} V$ for a legal recursive right prefix.

**Lemma 9.1** (Prefix deletion)**.** *An actual native $V$ reaches every legal recursive right prefix $P$ by finitely many indices in $\{0, 1\}$.*

*Proof.* To remove complete trailing private columns, use index zero in a general case; in a special case use the zero or one step which keeps one enclosing entry as required. Each such operation decreases the finite column-and-entry count and retains the desired prefix. To remove an entry

$u$ before a retained earlier same-column prefix, first clear its private height by this independent column procedure. It becomes a bare entry with parent $p$. It is not the singleton owner special case, because its column has a retained prefix (or is top-level). A general one step, followed by deletion of its newly trailing columns, replaces it with a finite root suffix whose parents are all less than $p$. Delete those entries by induction on the natural number $p$. Finally follow the finite right spine of $P$ with these two procedures. The inductions are on object count and parent address, never on the native descendant order. □

The native rightmost-copy formula has the following prefix barrier: if an expansion crosses below an intermediate descendant from a common finite source, its lower result must be a recursive prefix of that intermediate node. The proof compares the two finite paths to their first differing indices. The smaller-index result is a recursive prefix of the larger-index result. A subsequent jump below the intermediate value is governed by the same rightmost hole, so either the paths reduce to a common shorter pair, or that jump lands at a recursive prefix. Induction on the sum of the remaining path lengths terminates this argument. Section 9.2 gives the finite comparison argument; no infinite descent assertion is used.

**Lemma 9.2** (Reachability below an actual node)**.** *If $K, V \in \mathscr{U}$ and $K < V$, there is a nonempty finite native path $V \to^* K$.*

*Proof.* Let $N$ be the column-and-entry count of $K$. From the current source delete to its least legal recursive prefix $U > K$. Its first-difference description has at most $N + 2$ objects: beyond the common part with $K$ only the distinguishing entry and, if needed, its containing column are necessary. Prefix deletion is available by Lemma 9.1.

Test the finitely many children $U[0], \dots, U[N + 1]$. Equality succeeds. If a crossing below $K$ occurs without the child's being a prefix of $K$, it is a finite rejection witness. Otherwise a child above $K$ gives a new compressed prefix, strictly below the old compressed prefix. The finite child bound follows from the native adjacent-prefix identities: if all those children were below $K$ without such a crossing, they would give too many distinct prefixes of the $N$-object target. The empty-last-column successor is dealt with by its deletion child.

Compressed states have at most $N + 2$ objects and scope-bounded natural parent addresses. There are only finitely many such syntactic states. Their strictly decreasing comparison therefore makes this algorithm terminate, without assuming the entire comparison well-founded. The state bound depends on the fixed target alone, as shown in Section 9.2.

A rejection cannot occur for actual $K, V$. Their finite initial towers are recursive-prefix comparable; prefix deletion puts their two given histories under one finite common source. The finite path prefix barrier then excludes exactly the purported rejecting crossing. Thus the algorithm succeeds, producing the required finite path. It is nonempty because $K < V$. □

## 9.2 The finite path barrier and the state bound

The reachability proof uses the following three direct identities of the original rightmost operation. For every legal recursive prefix $P \prec_{\mathrm{pref}} X$,

$$X[n] \preceq_{\mathrm{pref}} X[n+1], \qquad P \preceq_{\mathrm{pref}} X[n] \text{ or } X[n] \preceq_{\mathrm{pref}} P, \qquad P \preceq_{\mathrm{pref}} X[1]. \tag{9.1}$$

To verify them, locate the cut of $P$ on the rightmost active branch. The general zero case deletes the deepest last column. A positive case keeps the prefix before its final bare entry and fills only the rightmost hole; increasing the index fills that same hole further. An ancestor root does not rewrite the retained prefix. The special case deletes the singleton owner-bare column and appends copies of the same shortened enclosing entry; index one keeps at least one such entry. If index zero removes an enclosing entry entirely, its output is below the corresponding prefix cut.

Empty-last-column deletion is immediate. Comparing the cut with these positions proves (9.1) by finite recursion on the containing branch.

If $P \preceq_{\mathrm{pref}} X$ and a given finite path from $X$ ends at $Y > P$, then $P \prec_{\mathrm{pref}} Y$: follow (9.1) edge by edge. Once a path reaches $P$ or below, strict native decrease prevents its return above $P$. At the first crossing below $P$, its output is a recursive prefix of $P$.

**Lemma 9.3** (Two finite paths)**.** *Suppose $X, Y$ are recursive-prefix comparable, with given finite paths $X \to^* H$ and $Y \to^* B$. If $P = H[n]$ and $P < B < H$, then $P \prec_{\mathrm{pref}} B$.*

*Proof.* Induct on the sum of the remaining lengths of these two given paths, keeping $H, B, P$ fixed. If $X = Y = H$, the first child $V$ on the path to $B$ is above $P$. The children of $H$ are prefix comparable by (9.1), so $P \prec_{\mathrm{pref}} V$, and the preceding finite-path barrier gives $P \prec_{\mathrm{pref}} B$. If $X = Y \neq H$, neither endpoint is the common starting point: $B < H \leq X$. Take the first edge on both paths; the new starting points are children of one node and hence prefix comparable. The remaining total length decreases.

If $X \prec_{\mathrm{pref}} Y$, then $B < H \leq X$. On the given path from $Y$ to $B$, take its first point $Y' \leq X$. The barrier gives $Y' \preceq_{\mathrm{pref}} X$. At least one edge has been consumed, so apply the induction hypothesis to the remaining paths. If $Y \prec_{\mathrm{pref}} X$ and $H > Y$, the path barrier first gives $Y \prec_{\mathrm{pref}} H$. Since $P < B \leq Y$, (9.1) at $H \to P$ forces $P \prec_{\mathrm{pref}} Y$, and the path to $B$ gives the conclusion. If instead $H \leq Y$, take the first $X' \leq Y$ on the path from $X$ to $H$. Again $X' \preceq_{\mathrm{pref}} Y$ and a positive number of edges is consumed. All unfinished branches strictly reduce the stated natural-number measure. No well-ordering of the descendant relation is assumed. □

For the size bound, the legal recursive prefixes of a finite $K$ form a chain of at most $\nu(K) + 1$ members: each possible stopping place corresponds to a column or entry in depth-first order, and invalid empty private-column stops are omitted. If $K < X$ is not itself a prefix, let $\rho_K(X)$ be the least legal recursive prefix of $X$ above $K$. At the first difference of $X$ and $K$, retain just that difference and truncate everything later. A differing parent needs its entry with empty height. If $K$ ends first, retain one additional entry, one additional top-level empty column, or a private column and its first bare entry. Thus

$$K < \rho_K(X) \preceq_{\mathrm{pref}} X, \qquad \nu(\rho_K(X)) \leq \nu(K) + 2. \tag{9.2}$$

The earlier containing framework was already counted in $K$.

Put $N = \nu(K)$. If a compressed state $U$ is a limit type, its $N + 2$ children with indices $0, \ldots, N + 1$ are distinct: each additional general copy adds a nonempty native window, and each special copy adds a shortened entry. If they were all below $K$ without a rejecting nonprefix crossing, they would be $N + 2$ distinct strict prefixes of $K$, contradicting its prefix bound. For a successor type, the deletion child is the immediate predecessor, so it is at least $K$. This proves the finite child bound used in Lemma 9.2.

Finally, every parent in a scope-correct expression of at most $N + 2$ objects refers to a visible earlier column; its address is bounded by the total number of columns. There are finitely many tree shapes and such address assignments. Compressed states therefore range over a fixed finite set and strictly decrease, proving termination of the search. If a rejecting crossing occurred for actual $K$ and $V$, their given histories start at prefix-comparable towers; apply Lemma 9.3 to that crossing to contradict rejection. This establishes reachability without using any ordinal labels.

## 9.3 Least maximum labels

*Proof of Theorem 1.1.* Fix $g'$ from Theorem 8.2. Every actual $M$ has a finite history from some $I_k$. Ordinary induction on that finite history, using Theorem 7.2, supplies at least one legal labelling of $M$ with maximum below $g'$. Define externally in ZFC

$$r(M) = \min\{\alpha < g' : \exists C\, [C \text{ legal on } C_{\mathrm{original}}(M) \ \wedge \ \rho(C) = \alpha]\}. \tag{9.3}$$

The set exists by Separation on the ordinal $g'$ and is nonempty, so its least member exists. This is not minimization inside a weak endpoint. All the graph and small-structure clauses have their actual external set-theoretic meanings.

Choose a legal $C$ attaining $r(M)$. The arbitrary-source quantifier in Theorem 7.2 makes it applicable to this $C$, and gives

$$r(M[n]) \leq \rho(C') < r(M). \tag{9.4}$$

Each node's rank is defined independently by (9.3). A previous step's specially chosen output is not an extra eligibility condition on a later source. The physical-source identity required by the construction was established by replay, not by naming an output as a new input.

For actual $K < M$, Lemma 9.2 supplies a nonempty finite path from $M$ to $K$. Apply (9.4) along it to obtain $r(K) < r(M)$. Thus $r$ strictly embeds the original comparison into $g'$. More directly, a nonempty subset of $\mathscr{U}$ has a least rank in its rank image; a node attaining it has no smaller member of that subset. Hence the original linear comparison is a well-order, not merely a relation with terminating selected fundamental sequences. Finally put $r(\text{Limit}) = \iota_2$. By (8.2), $r(I_n) < g' < \iota_2$ for every $n$, and all subsequent edges decrease by (9.4). This proves every assertion. □

## Source notes and attribution

This is an edited condensation of the v139 continuation manuscript and its earlier notebook, with the same pinned native BTBMS operation. Historical progress reports, superseded restricted cases and bounded experimental logs are omitted. The principal arguments are included in the text; no numbered notebook section is a necessary external proof reference. The accompanying source concordance records the origin of the consolidated arguments and the input hashes.

The three prior amendments remain explicit: the actual work model retains its required elementary relations; the empty-tail-prefix interval uses the entry count $\mu$; and a proper generated restriction assumes source Gen. Low coding of actual incoming graphs is shortened using that same source Gen and the independently proved finite-term transport. Its scope is the actual legal maps used in this proof; it is not a claim that an arbitrary weak cut has a canonical graph.

Standard background comprises constructibility, small-structure satisfaction, admissible Collection, condensation and the Silver-indiscernible shift obtained from $0^{\#}$. The following sources are background references, not published proofs or endorsements of the BTBMS argument:

[1] Klaus Sutner, *The Lattice of $\beta$ Recursively Enumerable Sets*, Section 2.2. https://www.cs.cmu.edu/~sutner/papers/beta.pdf

[2] Victoria Gitman, Joel David Hamkins and Thomas A. Johnstone, *What is the theory ZFC without power set?* https://victoriagitman.github.io/files/ZFC-.pdf

[3] Victoria Gitman and P. D. Welch, *Ramsey-like Cardinals II*. https://arxiv.org/abs/1104.4448

[4] Noam Greenberg, *Two Applications of Admissible Computability*. https://homepages.ecs.vuw.ac.nz/~greenberg/Papers/72-Applications_of_higher_computability.pdf

The user reports that the originator of F circulated a separate proof on QQ and acknowledged the independent development of G. An exact public date and stable public link have not been established for that communication. The acknowledgment is recorded as one of independence, not as verification of this manuscript. The initial idea of G is attributed to the user; the subsequent AI-assisted arguments are not presented as the user's unaided work.

# A The pinned native operation

The following excerpt fixes the finite data, relative-height comparison, root window and both expansion branches. Addresses in the code are zero-based; displayed addresses in the mathematical text are one-based. The comparator helpers implement the recursive lexicographic order of Section 1; the full pinned file is included in the source package. This is a definition, not a finite test offered as a proof.

```
export type Expr = Column[];
export type Column = Entry[];
export type Entry = { value: number; height: Expr };

export type ExprData<T extends object> = ColumnData<T>[];
export type ColumnData<T extends object> = EntryData<T>[];
export type EntryData<T extends object> = T & { height: ExprData<T> };

export type Height = ExprData<{ value: number; mark: boolean }>;
export type Vertical = Height[];

export const INFINITY: Expr = Infinity as any;
export const INFINITY_height: Height = Infinity as any;

export function is_infinity(e: Expr | Height): boolean {
    return e === INFINITY;
}

export function infinity_FS(index: number): Expr {
    if (index === 0) return [[]];
    let col: Column = [{ value: index - 1, height: [] }];
    for (let i = index - 1; i > 0; i--) {
        col = [{ value: i - 1, height: [col] }];
    }
    return [[], col];
}

export function to_height(e: Expr, r: number): Height {
    const result: Height = [];

    for (let i = 0; i < e.length; i++) {
        const col = e[i];

        const result_col: Height[number] = [];

        for (const entry of col) {
            const p = entry.value;
            let mark: boolean, value: number;
            if (p < r) {
                [mark, value] = [false, p];
            } else {
                [mark, value] = [true, p - r];
            }
            result_col.push({
                mark,
                value,
                height: to_height(entry.height, r),
            });
        }

        result.push(result_col);
    }

    return result;
}
```

```
56
57 export function from_height(h: Height, r: number): Expr {
58     const result: Expr = [];
59
60     for (let i = 0; i < h.length; i++) {
61         const col = h[i];
62
63         const result_col: Column = [];
64
65         for (const entry of col) {
66             const { mark, value } = entry;
67             result_col.push({
68                 value: value + (mark ? r : 0),
69                 height: from_height(entry.height, r),
70             });
71         }
72
73         result.push(result_col);
74     }
75
76     return result;
77 }
78
79 export function height_compare(a: Height, b: Height): number {
80     if (is_infinity(a) || is_infinity(b)) return boolean_compare(is_infinity(a), is_infinity(b
       ));
81     return lex_compare(a, b, lex_compare_by(height_entry_comparator));
82 }
83
84 const height_entry_comparator = object_lex_compare_by(
85     { mark: boolean_compare, value: number_compare, height: height_compare },
86     ['mark', 'value', 'height'],
87 );
88
89 export function vertical_compare(a: Vertical, b: Vertical): number {
90     return lex_compare(a, b, height_compare);
91 }
92
93 export function has_next_layer<T extends object>(expr: ExprData<T>): boolean {
94     const right = expr.length - 1;
95     if (right === -1) return false;
96     const top = expr[right].length - 1;
97     return top !== -1;
98 }
99
100 export function next_layer<T extends object>(expr: ExprData<T>): ExprData<T> {
101     const right = expr.length - 1;
102     const top = expr[right].length - 1;
103     return expr[right][top].height;
104 }
105
106 export function skip_layers<T extends object>(expr: ExprData<T>, l: number): ExprData<T> {
107     for (let i = 0; i < l; i++) expr = next_layer(expr);
108     return expr;
109 }
110
111 export function tail_layer(expr: Expr): number {
112     if (!has_next_layer(expr)) return -1;
113     return 1 + tail_layer(next_layer(expr));
114 }
115
116 export function tail(expr: Expr, t_layer: number): number {
117     let current_left = 0;
118     let current = expr;
```

```
    for (let i = 0; i < t_layer; i++) {
        current_left += current.length;
        current = next_layer(current);
    }
    return current_left + current.length - 1;
}

export function root(expr: Expr, t_layer: number): number {
    const current = skip_layers(expr, t_layer);
    return current[current.length - 1][current[current.length - 1].length - 1].value;
}

export function root_layer(expr: Expr, r: number): [l: number, ri: number] {
    let current = expr;
    let current_left = 0;
    let current_layer = 0;
    while (true) {
        if (r < current_left + current.length) {
            return [current_layer, r - current_left];
        }
        current_left += current.length;
        current = next_layer(current);
        current_layer++;
    }
}

function ascend_replace(expr: Expr, r: number, diff: number, t_layer: number | undefined,
    new_tail: Expr): Expr {
    let result: Expr = [];

    for (let i = 0; i < expr.length; i++) {
        if (t_layer === 0 && i === expr.length - 1) {
            result.push(...new_tail);
        } else {
            const col = expr[i];
            let result_col: Column = [];
            for (let j = 0; j < col.length; j++) {
                const entry = col[j];
                const new_t_layer =
                    t_layer !== undefined && i === expr.length - 1 && j === col.length - 1 ?
    t_layer - 1 : undefined;
                result_col.push({
                    value: entry.value >= r ? entry.value + diff : entry.value,
                    height: ascend_replace(entry.height, r, diff, new_t_layer, new_tail),
                });
            }
            result.push(result_col);
        }
    }

    return result;
}

export function is_special(expr: Expr, t_layer: number): boolean {
    if (t_layer === 0) return false;
    let current = expr;
    let current_left = 0;
    for (let i = 0; i < t_layer; i++) {
        current_left += current.length;
        current = next_layer(current);
    }
    if (current[current.length - 1].length !== 1) return false;
    const entry = current[current.length - 1][0];
    return entry.height.length === 0 && entry.value === current_left - 1;
```

```
}

export function expand_special(expr: Expr, t_layer: number, index: number): Expr {
    let result = expr.slice(0, -1);
    let col = expr[expr.length - 1];
    let result_col = col.slice(0, -1);
    let entry = col[col.length - 1];
    if (t_layer > 1) {
        let new_entry = {
            value: entry.value,
            height: expand_special(entry.height, t_layer - 1, index),
        };
        result_col.push(new_entry);
    } else {
        let new_entry = {
            value: entry.value,
            height: entry.height.slice(0, -1),
        };
        if (result_col.length === 0) result_col.push(new_entry);
        result_col.push(...Array<Entry>(index).fill(new_entry));
    }
    result.push(result_col);
    return result;
}

export function root_appending_start(col_root: Column, r: number, col_tail: Column, t: number):
     number {
    let heights_root = col_root.map(({ height }) => to_height(height, r));
    let heights_tail = col_tail.slice(0, -1).map(({ height }) => to_height(height, t));
    let ir = 0,
        it = 0;
    while (ir !== heights_root.length && it !== heights_tail.length) {
        const cmp = height_compare(heights_root[ir], heights_tail[it]);
        if (cmp >= 0) it++;
        if (cmp <= 0) ir++;
    }
    return ir;
}

export function is_limit(expr: Expr): boolean {
    return is_infinity(expr) || (expr.length > 0 && expr[expr.length - 1].length > 0);
}

function FS(expr: Expr, index: number): Expr {
    if (is_infinity(expr)) return infinity_FS(index);
    if (expr.length === 0) return expr;
    const t_layer = tail_layer(expr);
    if (t_layer < 0) return expr.slice(0, -1);

    if (is_special(expr, t_layer)) {
        return expand_special(expr, t_layer, index);
    }

    const t = tail(expr, t_layer);
    const r = root(expr, t_layer);
    const [r_layer, ri] = root_layer(expr, r);

    const expr_root = skip_layers(expr, r_layer);
    const col_root = expr_root[ri];
    const expr_tail = skip_layers(expr_root, t_layer - r_layer);
    const col_tail = expr_tail[expr_tail.length - 1];

    const appending = root_appending_start(col_root, r, col_tail, t);

```

```
244     let new_tail: Expr = [];
245
246     for (let j = index; j >= 1; j--) {
247         if (ri !== expr_root.length - 1) {
248             let new_tail_1 = ascend_replace(expr_root.slice(ri + 1), r, j * (t - r), t_layer -
        r_layer, new_tail);
249             let new_col = ascend_replace([col_tail.slice(0, -1)], r, (j - 1) * (t - r),
        undefined, [])[0];
250             for (let k = appending; k < col_root.length; k++) {
251                 new_col.push({
252                     value: col_root[k].value,
253                     height: ascend_replace(col_root[k].height, r, j * (t - r), undefined, []),
254                 });
255             }
256             new_tail = [new_col, ...new_tail_1];
257         } else {
258             if (appending === col_root.length) throw new Error('Illegal state');
259             let new_col = ascend_replace([col_tail.slice(0, -1)], r, (j - 1) * (t - r),
        undefined, [])[0];
260             for (let k = appending; k < col_root.length; k++) {
261                 new_col.push({
262                     value: col_root[k].value,
263                     height: ascend_replace(
264                         col_root[k].height,
265                         r,
266                         j * (t - r),
267                         k === col_root.length - 1 ? t_layer - r_layer - 1 : undefined,
268                         new_tail,
269                     ),
270                 });
271             }
272             new_tail = [new_col];
273         }
274     }
275
276     return ascend_replace(expr, 0, 0, t_layer, new_tail);
277 }
```